
PROYECTO MODULAR · INGENIERÍA INFORMÁTICA

Sistema de Predicción Ex-Ante del Impacto de Desastres Naturales en México

Manual técnico del proyecto modular

Autor: Alan Pérez Pelayo

Fecha: 6 de septiembre de 2026

Repositorio: sacalvario/Desastres-naturales (rama `main`)

Documento técnico de referencia. Describe la arquitectura del sistema, justifica cada decisión de modelado, delimita el alcance de lo que el sistema puede y no puede responder, y enumera sus limitaciones junto con la mejora concreta que levantaría cada una. Todas las cifras se obtuvieron ejecutando código sobre los archivos reales del repositorio.

Contenido

1. Resumen ejecutivo

2. Problema y justificación

- El costo de los desastres en México
- Por qué importa estimar antes
- El hueco que llena
- Naturaleza académica del documento

3. Arquitectura del sistema

- Vista general
- La frontera entre entrenamiento y servicio
- Inventario de artefactos
- Flujo de una petición de predicción

4. Los datos

- Origen
- Esquema de Base.xlsx
- Nombres de columna con acentos: son parte del contrato
- Limpieza: qué hace `clean_base()` y por qué
- La tabla de población: interpolación censal
- Lectura defensiva del Excel
- Las dos capas de filtrado: `stats_data` frente a `data`
- Perfil estadístico del objetivo
- Distribución de las variables categóricas
- Cobertura temporal
- Los diez eventos más costosos de la base
- Los diez estados con mayor daño acumulado

5. El modelo de aprendizaje automático

- Qué se está modelando exactamente
- Por qué `HistGradientBoostingRegressor`
- Por qué dos modelos independientes con un preprocesador compartido
- Por qué la transformación logarítmica del objetivo
- Por qué las variables cíclicas del mes
- Fuga de datos: el punto fuerte del proyecto
- Split temporal en lugar de aleatorio
- Interpretación honesta de las métricas

6. La API

- Marco y arranque
- Dependencias fijadas
- GET / — sonda mínima
- GET /health — diagnóstico
- POST /predict — el endpoint principal
- Los ocho endpoints de estadísticas
- Cacheo de los endpoints de estadísticas
- CORS: lista blanca en producción, cualquier puerto local en desarrollo

7. El frontend

- Pila tecnológica
- Estructura y las dos pestañas
- Pestaña 1: el predictor
- Pestaña 2: el dashboard histórico
- El mapa coroplético
- Configuración de entorno

8. Cómo se ejecuta

- Requisitos previos
- Puesta en marcha del backend
- Reentrenamiento de los modelos
- Puesta en marcha del frontend
- Orden de arranque
- Despliegue

9. Alcance del sistema

- Lo que el sistema SÍ hace
- Lo que el sistema NO hace
- La formulación precisa de lo que responde

10. Limitaciones y áreas de oportunidad

- L1. Tamaño y desbalance del conjunto de datos
 - L2. Granularidad estatal, no municipal
 - L3. Ausencia de intervalos de confianza
 - L4. Ausencia de variables físicas del evento
 - L5. Daños en pesos nominales, sin deflactar
 - L6. Cobertura de pruebas parcial — parcialmente resuelta
 - L7. Sin versionado de modelos ni reentrenamiento programado
 - L8. Carga de artefactos en el import del módulo, sin manejo de fallos
 - L9. Sin validación de dominio en las entradas
 - L10. Riesgo latente de desalineación de nombres de estado
 - L11. Sesgo de retransformación: subestimación sistemática
 - L12. Datos simulados en el dashboard — resuelta
 - L13. Sin optimización de hiperparámetros
 - L14. Simplificación de eventos multiestado
 - L15. Ceros ambiguos en Población afectada
 - L16. Escala lineal de color en el mapa — resuelta
 - L17. Deuda técnica menor pero real
- Qué haría falta para llevarlo a producción real

11. Glosario

12. Referencias

- Fuentes de datos
- Bibliotecas y herramientas
- Fundamentos metodológicos
- Documentación interna del proyecto
- Nota sobre la verificación de las cifras

CAPÍTULO 1

Resumen ejecutivo

Este documento describe, justifica y delimita un sistema web de aprendizaje automático que estima **de forma anticipada (ex-ante)** dos magnitudes del impacto de un desastre natural en México:

1. el **daño económico** en millones de pesos, y
2. la **población afectada** en número de personas.

"Ex-ante" significa que la estimación se produce **antes de que el daño sea conocido y reportado**: el sistema solo recibe información disponible con anterioridad al evento —el tipo de fenómeno, el estado de la República, el mes y el año— y devuelve el impacto esperado. No usa ninguna variable que solo pueda medirse después del desastre.

El sistema se compone de tres piezas:

Pieza	Tecnología	Función
Entrenamiento	Python 3.11, pandas, scikit-learn	Lee <code>Base.xlsx</code> y <code>Poblacion_01.xlsx</code> , limpia, normaliza, entrena y serializa artefactos <code>.joblib</code>
API	FastAPI + Uvicorn	Carga los artefactos y expone <code>/predict</code> , <code>/health</code> y ocho endpoints <code>/stats/*</code>
Interfaz	React 19 + Vite	Dos pestañas: formulario predictor y dashboard histórico interactivo con filtrado cruzado y mapa coroplético

La base histórica contiene **3 969 registros de eventos** entre **2000 y 2023**, que tras la limpieza quedan en **3 958 filas útiles** repartidas en los **32 estados** del país, **2 clasificaciones** de fenómeno y **9 tipos**. El daño económico acumulado en esa base asciende a **668 035.68 millones de pesos** y la población afectada acumulada a **58 897 609 personas**.

Los modelos son dos `HistGradientBoostingRegressor` independientes que comparten un mismo preprocesador y entrenan sobre el logaritmo del objetivo. Evaluados con un **split temporal honesto** (entrenamiento con años anteriores a 2020, prueba con 2020 en adelante), obtienen:

Modelo	R ² en escala log	MAE	Error absoluto mediano	n entrena / n prueba
Daño económico	0.1661	209.09 MDP	1.69 MDP	3 226 / 732
Población afectada	0.1033	4 380.21 personas	49.62 personas	3 191 / 732

Estas cifras son modestas, y el manual **no las disfraza**: el capítulo 6 explica por qué son las cifras correctas a reportar y el capítulo 11 documenta, una por una, las limitaciones que las explican junto con la mejora concreta que levantaría cada techo. El valor defendible del proyecto no está en la magnitud del R², sino en la **corrección metodológica** con que se obtuvo: sin fuga de información, con validación temporal y con una arquitectura reproducible de extremo a extremo.

CAPÍTULO 2

Problema y justificación

El costo de los desastres en México

México está expuesto simultáneamente a dos familias de amenaza. Por un lado, la **geológica**: se asienta sobre la interacción de cinco placas tectónicas y alberga volcanes activos. Por otro, la **hidrometeorológica**: recibe ciclones tropicales por sus dos litorales, sufre sequías recurrentes en el norte y heladas en el altiplano. La base de datos de este proyecto cuantifica la consecuencia: **668 035.68 millones de pesos** de daño registrado en 24 años, concentrados de manera extremadamente desigual —el **1 % de los eventos más costosos concentra el 66.7 % del daño total**, y los **diez eventos más caros solos representan el 45 %**.

Por qué importa estimar *antes*

La utilidad de una estimación de impacto depende por completo del momento en que se produce.

Estimar después del evento es contabilidad. Es indispensable —CENAPRED lo hace y de ahí sale la base de datos que aquí se usa— pero llega cuando las decisiones ya se tomaron.

Estimar antes del evento es planeación, y habilita al menos tres usos:

1. **Protección civil y preposicionamiento.** Si en octubre un ciclón se dirige a Guerrero, saber que la clase de eventos "ciclón en Guerrero en octubre" tiene un impacto esperado de cientos de millones de pesos —frente a una decena de millones para "ciclón en Guerrero en junio"— permite decidir dónde colocar albergues, insumos y personal antes de que el fenómeno toque tierra.
2. **Presupuestación de instrumentos financieros.** El extinto FONDEN (Fondo de Desastres Naturales) y el FONREC (Fondo de Reconstrucción de Entidades Federativas) son mecanismos que requieren dimensionar reservas con anticipación. La misma lógica aplica hoy al ramo presupuestal que absorbió sus funciones y a los bonos catastróficos que México ha emitido. Un modelo que agrega el impacto esperado por estado, tipo de fenómeno y temporada es un insumo directo para ese dimensionamiento.
3. **Planeación territorial y priorización de obra.** Un mapa que muestre qué estados acumulan más daño y de qué fenómeno permite discutir dónde invertir en infraestructura de mitigación.

El hueco que llena

La información sobre desastres en México existe, pero está fragmentada y en un formato que no es consumible por un sistema:

- CENAPRED publica el impacto socioeconómico anual **en PDF**, evento por evento, sin una interfaz de consulta programática.
- INEGI publica el censo de población **en tabulados de Excel** con encabezados de varias filas.

- No existe un punto de acceso público que responda a la pregunta operativa "*¿cuánto impacto espero de un fenómeno de tipo X, en el estado Y, en el mes Z?*".

Este proyecto no genera datos nuevos: **integra** los que existen, los **normaliza** hasta hacerlos utilizables, entrena un modelo sobre ellos y los **publica** por una API y una interfaz web. La contribución es de ingeniería de datos y de método, no de recolección.

Naturaleza académica del documento

Este es un **proyecto modular de la carrera de Ingeniería Informática**. Debe sostenerse ante una evaluación, lo que implica que cada decisión técnica lleve su justificación y que los límites del sistema estén enunciados por el propio autor antes de que alguien más los señale. Ese es el criterio con el que está escrito el manual: no describe únicamente qué hace el sistema, sino **por qué está hecho así y qué no hace**.

CAPÍTULO 3

Arquitectura del sistema

Vista general

El sistema es una tubería lineal en cuatro etapas, con un punto de corte deliberado entre las dos primeras y las dos últimas.

FUENTES (Excel)

```

Base.xlsx
3 969 eventos
2000-2023
Poblacion_01.xls
censo INEGI
2000/05/10/20

```

ENTRENAMIENTO (offline)

```

backend/train_model_simple.py
1. _read_excel_safe()
2. clean_base()
3. load_population_table()
4. evaluate_temporal()
5. fit_final()
6. joblib.dump(...)

```

FRONTERA: aquí termina la lectura de Excel

ARTEFACTOS backend/app/artifacts/

```

model_danos.joblib 370 KB
model_poblacion.joblib 370 KB
preprocessor.joblib 4.0 KB
poblacion_estatal.joblib 14 KB
data.joblib 2.1 MB
metadata.json 1.0 KB

```

joblib.load() al importar

API (runtime)

```

backend/app/main.py (FastAPI)
POST /predict
GET /health
GET /stats/kpis
GET /stats/evolucion-anual
GET /stats/top-estados
GET /stats/por-estado
GET /stats/clasificacion
GET /stats/top-eventos

```

HTTP + JSON (CORS allowlist)

INTERFAZ

```

frontend/ React 19 + Vite
App.jsx ..... predictor
DashboardHistorico.jsx . panel
api.js ..... fetch
public/mexico.geojson . 360 KB

```

La frontera entre entrenamiento y servicio

La línea doble del diagrama es la decisión arquitectónica más importante del proyecto: **la API nunca lee los archivos de Excel**. Todo lo que necesita en tiempo de ejecución vive en `backend/app/artifacts/`, y todo lo que hay ahí lo produjo `train_model_simple.py`.

Cuatro razones lo justifican:

- 1. Rendimiento y latencia.** Abrir y parsear un `.xlsx` de 1 MB con `openpyxl` tarda del orden de segundos; deserializar un `DataFrame` con `joblib` tarda milisegundos. Si la API leyera el Excel en cada petición, cada gráfica del dashboard costaría segundos. Si lo leyera una sola vez al arrancar, seguiría pagando ese costo en cada despliegue y en cada reinicio de un contenedor.
 - 2. Determinismo y reproducibilidad.** Si el Excel se lee en runtime, dos instancias de la API arrancadas en momentos distintos podrían servir números distintos porque alguien editó una celda entretanto. Con artefactos, **el estado del sistema es un conjunto de archivos versionables**: el modelo servido y los datos del dashboard son exactamente los que produjo la última corrida del entrenamiento, ni más ni menos.
 - 3. Separación de responsabilidades.** Toda la lógica de limpieza —conversión de tipos, normalización de estados, interpolación de población, cálculo de variables cíclicas— vive en un solo archivo. `main.py` no sabe qué es `"Varios Estados"` ni que el censo dice `"Coahuila de Zaragoza"`. Esa lógica no está duplicada en dos sitios, que es la fuente clásica de divergencias entre lo que el modelo aprendió y lo que la API le entrega.
 - 4. Despliegue.** El servidor de producción no necesita los archivos de Excel, ni `openpyxl` funcionando, ni permisos sobre la ruta donde viven las fuentes. Solo necesita el directorio `artifacts/`.
- Consecuencia operativa que hay que tener presente:** editar `Base.xlsx` **no cambia nada** en la aplicación. Para que un cambio en las fuentes se refleje hay que volver a ejecutar `python train_model_simple.py` y volver a desplegar los artefactos. Es un compromiso consciente: se pierde inmediatez a cambio de determinismo.

Inventario de artefactos

Archivo	Tamaño	Contenido	Consumido por
<code>model_danos.joblib</code>	370 KB	<code>HistGradientBoostingRegressor</code> del daño económico	<code>/predict</code>
<code>model_poblacion.joblib</code>	370 KB	<code>HistGradientBoostingRegressor</code> de población afectada	<code>/predict</code>
<code>preprocessor.joblib</code>	4.0 KB	<code>ColumnTransformer</code> ya ajustado (one-hot + passthrough)	<code>/predict</code>
<code>poblacion_estatal.joblib</code>	14 KB	Diccionario <code>{(Estado, Año): población}</code> + metadatos de rango	<code>/predict</code>
<code>data.joblib</code>	2.1 MB	<code>DataFrame</code> limpio de $3\,958 \times 31$ con el histórico	los seis <code>/stats/*</code>
<code>metadata.json</code>	1.0 KB	Modo, features, métricas temporales, tipo de modelo	<code>/health</code>

Obsérvese que `data.joblib` **no** es el conjunto con el que se entrena: es un superconjunto, por razones que se explican en el capítulo 5.

Flujo de una petición de predicción

1. El usuario llena el formulario en `App.jsx` (año, mes, clasificación, tipo, estado) y envía.
2. `api.js` hace `POST {VITE_API_URL}/predict` con un JSON cuyos campos usan **guion bajo** en lugar de espacio.
3. FastAPI valida el cuerpo contra el modelo Pydantic `PredictRequest`.
4. `predict()` normaliza el nombre del estado, consulta la población estatal en el diccionario, calcula `mes_sin` y `mes_cos`, y arma un `DataFrame` de una fila con los **nombres de columna con espacios** que el preprocesador espera.
5. El preprocesador transforma esa fila a un vector de 47 números.
6. Cada modelo predice en escala logarítmica; la API aplica `expm1` y trunca en cero.
7. Devuelve `{"prediction": {...}}` y React pinta dos tarjetas.

CAPÍTULO 4

Los datos

Origen

Archivo	Origen	Contenido	Dimensiones
Base.xlsx	CENAPRED (Centro Nacional de Prevención de Desastres) y fuentes asociadas: FONDEN, CENACOM, SAGARPA, CEPAL, prensa	Un registro por evento de desastre con sus daños reportados	3 969 filas × 28 columnas
Poblacion_01.xlsx	INEGI, tabulado "Población total por entidad federativa y grupo quinquenal de edad"	Población por estado en años censales	55 filas × 6 columnas (con 7 filas de encabezado)

El propio archivo de INEGI registra la fecha de consulta: **13/06/2026**. La columna **Fuente** de **Base.xlsx** contiene **66 valores distintos**, lo que revela que la base es una compilación de varios reportes: **CENAPRED**, **FONDEN**, **CENACOM**, **SAGARPA**, **PACC**, **FAPRACC**, combinaciones de ellos, y en algunos casos **Prensa** o **Reforma, Universal y Milenio**.

Esquema de Base.xlsx

Las 28 columnas del archivo original se agrupan en cuatro bloques:

Bloque de identificación temporal y tipológica (lo que el modelo puede usar):

Columna	Tipo original	Papel
Fecha de Inicio	fecha	no usada por el modelo
Fecha de Fin	fecha (23 nulos)	no usada
Mes	entero 1–12	origen de mes_sin / mes_cos
Duración días	entero	no usada
Año	entero 2000–2023	feature numérica
Clasificación del fenómeno	texto, 2 valores	feature categórica
Tipo de fenómeno	texto, 9 valores	feature categórica
Estado	texto, 38 valores crudos	feature categórica (tras normalizar)

Bloque descriptivo (no usado por el modelo): **Municipios Afectados** (2 436 valores distintos, formato libre) y **Descripcion general de los daños** (3 740 valores distintos, texto narrativo).

Bloque de consecuencias reportadas (todas son *posteriores* al evento): Defunciones , Población afectada , Viviendas dañadas , Escuelas , Hospitales , Comercios , Area de cultivo dañada / pastizales (h) y Total de daños (millones de pesos) .

Bloque de trabajo del analista original (residuos de hoja de cálculo, no usados): A , B , Impacto humano , Daños a infraestructura , Unnamed: 23 , Unnamed: 24 , Unnamed: 25 , Min , Max . Las tres columnas Unnamed: están prácticamente vacías (3 969, 3 969 y 3 964 nulos respectivamente).

Nombres de columna con acentos: son parte del contrato

Un punto que sorprende a quien lee el código por primera vez es que los nombres de columna en español, **con acentos y con espacios**, aparecen literalmente en tres capas distintas:

```
TARGET_DANOS = "Total de daños (millones de pesos)"
CAT_FEATURES = ["Clasificación del fenómeno", "Tipo de fenómeno", "Estado"]
NUM_FEATURES = ["Año", "mes_sin", "mes_cos", "Población estatal"]
```

Estos strings no son cosméticos. El `ColumnTransformer` de scikit-learn se ajusta **sobre nombres de columna**, y al serializarlo con `joblib` esos nombres quedan grabados dentro del artefacto. Cuando la API construye la fila de entrada, si escribiera "Clasificacion del fenomeno" sin acentos, el preprocesador no encontraría la columna y fallaría. Por eso el `CLAUDE.md` del repositorio dice que son *load-bearing*: **son el contrato entre el Excel, el modelo y la API**, y cambiarlos exige cambiarlos en los tres lugares a la vez.

En la práctica esto obliga a un detalle fácil de pasar por alto: en el Excel original la columna se llama "Población afectada " **con un espacio al final**, y la clasificación aparece como "Geológico " , también con espacio. La primera línea útil de `clean_base()` es precisamente lo que evita que eso rompa todo:

```
df.columns = [c.strip() for c in df.columns]
```

y unas líneas después, para los valores:

```
for c in ["Clasificación del fenómeno", "Tipo de fenómeno", "Estado"]:
    df[c] = df[c].astype(str).str.strip()
```

Sin esos dos `strip`, "Geológico " y "Geológico" serían dos categorías distintas para el one-hot encoder, y la API —que envía la versión sin espacio— caería siempre en la categoría equivocada.

Limpieza: qué hace `clean_base()` y por qué

Conversión numérica tolerante

```
df["Año"] = pd.to_numeric(df["Año"], errors="coerce")
df["Mes"] = pd.to_numeric(df["Mes"], errors="coerce")
df[TARGET_DANOS] = pd.to_numeric(df[TARGET_DANOS], errors="coerce")
df[TARGET_POBL] = pd.to_numeric(df[TARGET_POBL], errors="coerce")
```

`errors="coerce"` convierte a `NaN` todo lo que no sea número en lugar de lanzar excepción. Es necesario porque las columnas de consecuencias contienen literales `"SD"` y `"sd"` (sin dato) mezclados con enteros. El efecto medido es:

Columna	Filas que quedan en <code>NaN</code>
Total de daños (millones de pesos)	9
Población afectada	44

Esto explica por qué los dos modelos entrenan con conteos distintos: 3 958 filas para daños y 3 923 para población.

Normalización de nombres de estado

La columna `Estado` es la más sucia del archivo. Tiene **38 valores distintos** para 32 estados, porque mezcla tres problemas:

a) Celdas con varios estados. Ejemplos reales:

- "CDMX, Estado de Mexico, Morelos, Puebla, Tlaxcala"
- "Chihuahua, Durango, Estado de Mexico, Puebla, Sonora, Tlaxcala"
- "Tamaulipas, Veracruz"
- "Veracruz y Tamaulipas "

b) Alias. "CDMX" frente a "Ciudad de México".

c) Espacios sobrantes. "Campeche ", "Ciudad de México ".

El tratamiento son tres líneas:

```
df["Estado"] = df["Estado"].str.split(r",| y ", regex=True).str[0].str.strip()
df["Estado"] = df["Estado"].replace(STATE_ALIASES) # CDMX→Ciudad de México, Estado de México→México
df = df[~df["Estado"].isin(NON_STATES)].copy() # elimina "Varios Estados"
```

Es decir: **partir por coma o por " y ", quedarse con el primer estado, limpiar espacios, aplicar alias y descartar la etiqueta no geográfica "Varios Estados"**.

El resultado verificado es exactamente **32 estados**, los 32 de la República, sin residuos:

Aguascalientes, Baja California, Baja California Sur, Campeche, Chiapas, Chihuahua, Ciudad de México, Coahuila, Colima, Durango, Guanajuato, Guerrero, Hidalgo, Jalisco, México, Michoacán, Morelos, Nayarit, Nuevo León, Oaxaca, Puebla, Querétaro, Quintana Roo, San Luis Potosí, Sinaloa, Sonora, Tabasco, Tamaulipas, Tlaxcala, Veracruz, Yucatán, Zacatecas.

Qué se pierde con esta decisión. Quedarse con el primer estado de una lista es una simplificación con costo: un ciclón que afectó a Tamaulipas y Veracruz se atribuye completo a Tamaulipas, y el daño de Veracruz desaparece del mapa. En esta base el impacto es acotado —solo hay 5 valores multi-estado distintos y las filas "Varios Estados" eliminadas son 2— pero es una fuente de sesgo real que el capítulo 11 retoma con su propuesta de corrección.

Variables cíclicas del mes

```
df["mes_sin"] = np.sin(2 * np.pi * df["Mes"] / 12)
df["mes_cos"] = np.cos(2 * np.pi * df["Mes"] / 12)
```

Se explica en detalle en el capítulo 6.

La tabla de población: interpolación censal

El tabulado de INEGI trae la población de cada estado en **cuatro años censales**: 2000, 2005, 2010 y 2020. Pero los eventos ocurren en los 24 años de 2000 a 2023. Se necesita un valor para cada año.

`load_population_table()` resuelve así:

```
raw = _read_excel_safe(POBL_PATH, header=None)
cen = raw.iloc[6:].copy() # se saltan 6 filas de encabezado
cen.columns = ["Estado", "Grupo", "2000", "2005", "2010", "2020"]
cen = cen[cen["Grupo"].astype(str).str.strip() == "Total"].dropna(subset=["Estado"])
...
for year in range(2000, 2024):
    rows.append((r["Estado"], year, float(np.interp(year, census_years, ys))))
```

Tres cosas ocurren aquí:

1. **Se saltan las primeras 6 filas** porque el tabulado de INEGI trae el nombre de la institución, el título del cuadro, la fecha de consulta y un encabezado de dos niveles antes de los datos. El archivo se lee con `header=None` justamente para poder recortar por posición.
2. **Se filtra `Grupo == "Total"`** porque el tabulado desglosa por grupo quinquenal de edad; solo interesa el agregado estatal.
3. **`np.interp` interpola linealmente** entre años censales y, fuera del rango, **extrapola plano** (repite el valor del extremo). Es el comportamiento por omisión de `numpy.interp` y aquí es deliberado.

Un ejemplo concreto verificado, Jalisco:

Año	Población	Cómo se obtuvo
2000	6 322 002	censo
2003	6 580 069	interpolación 2000–2005
2005	6 752 113	censo
2010	7 350 682	censo
2015	7 849 417	interpolación 2010–2020
2020	8 348 151	censo
2021	8 348 151	extrapolación plana
2023	8 348 151	extrapolación plana

La tabla resultante tiene **768 filas** = 32 estados × 24 años.

Por qué interpolación lineal y no algo más sofisticado. La población estatal es una serie suave y monótona en escalas de cinco años; una recta entre censos captura la tendencia con error pequeño frente a la variabilidad del fenómeno que se quiere predecir. Un modelo demográfico con tasas de crecimiento, migración y proyecciones de CONAPO sería más exacto, pero el capítulo 6 documenta que esta variable resultó **prácticamente neutra** para el modelo de daños (es redundante con el one-hot del estado), así que refinarla no compraría nada.

Por qué extrapolación plana y no lineal. Extrapolar linealmente la tendencia 2010–2020 hasta 2023 daría un número quizá algo mejor, pero extrapolarse hacia 2026 o 2030 —años que el formulario acepta— produciría cifras cada vez más especulativas. Congelar el último valor conocido es la opción conservadora: el sistema nunca inventa demografía que no observó.

Alias del censo

Los nombres oficiales de INEGI no coinciden con los de `Base.xlsx`. Tres requieren traducción:

```
CENSUS_TO_BASE = {
    "Coahuila de Zaragoza": "Coahuila",
    "Michoacán de Ocampo": "Michoacán",
    "Veracruz de Ignacio de la Llave": "Veracruz",
}
```

La verificación es que tras el `merge`, **ceros filas quedan con Población estatal nula**: los 32 estados hicieron match. Si un alias faltara, ese estado entero perdería su población y el modelo lo trataría distinto.

Lectura defensiva del Excel

`_read_excel_safe()` merece una nota porque su presencia cuenta la historia del entorno donde nació el proyecto:

```
def _read_excel_safe(path, **kwargs):
    src = Path(path)
    tmp_dir = Path(tempfile.mkdtemp())
    tmp_file = tmp_dir / src.name
    try:
        shutil.copy2(src, tmp_file)
    except PermissionError:
        subprocess.run(["robocopy", str(src.parent), str(tmp_dir), src.name,
                        "/R:0", "/W:0"], capture_output=True)
    ...
```

Copia el archivo a un directorio temporal antes de leerlo y, si el sistema operativo niega el permiso, recurre a `robocopy`. La razón es que en Windows, con el archivo abierto en Excel o sincronizándose por OneDrive, `pandas.read_excel` falla con `PermissionError`. Copiarlo primero esquivaría el bloqueo.

Nota de portabilidad relevante para este proyecto: `robocopy` es un ejecutable exclusivo de Windows. En Linux —el entorno de trabajo actual— esa rama nunca se ejecutará; si `shutil.copy2` fallara, `subprocess.run` lanzaría `FileNotFoundError` en lugar del `PermissionError` descriptivo que el código pretende dar. En la práctica no se ha observado porque en Linux la copia funciona, pero es un fragmento atado a un sistema operativo que ya no es el de desarrollo.

Las dos capas de filtrado: `stats_data` frente a `data`

Esta es una de las decisiones de diseño que más conviene poder explicar en la defensa, porque parece redundante y no lo es.

Del mismo `DataFrame` limpio salen **dos subconjuntos con criterios distintos**:

```
# Capa 1 – LAXA: dataset del dashboard
stats_data = df.dropna(subset=[TARGET_DANOS, "Año"]).copy()
stats_data = stats_data[stats_data[TARGET_DANOS] >= 0].copy()
joblib.dump(stats_data, art_dir / "data.joblib")

# Capa 2 – ESTRICTA: dataset de entrenamiento (dentro de fit_final / evaluate_temporal)
d = df.dropna(subset=FEATURES + [target]).copy()
```

	stats_data (dashboard)	data (entrenamiento)
Exige	objetivo y Año no nulos, objetivo ≥ 0	todas las features y el objetivo no nulos
Filas resultantes	3 958	3 958 (daños) / 3 923 (población)
Se guarda como	<code>data.joblib</code>	no se guarda; vive solo en memoria durante el entrenamiento
Lo consume	los endpoints <code>/stats/*</code>	<code>HistGradientBoostingRegressor.fit()</code>

Por qué deben existir las dos. Son dos preguntas distintas:

- El **dashboard** responde "*¿qué ha pasado históricamente?*". Es contabilidad. Una fila cuyo daño está reportado debe contar en el total nacional aunque le falte algún dato auxiliar; excluirla **subestimaría la realidad**. El dashboard debe ser lo más fiel posible a lo reportado.
- El **modelo** responde "*¿qué esperar de un evento futuro?*". Para aprender, cada fila necesita el vector de entrada **completo**: si le falta el mes, no hay `mes_sin`; si le falta la población estatal, hay un hueco. `scikit-learn` no puede ajustar con `NaN` en la entrada del `OneHotEncoder`, y aunque pudiera, imputar valores inventaría información.

Si se colapsaran en una sola capa habría que elegir un mal:

- Usar el filtro estricto para todo → el dashboard reportaría **menos daño del que realmente ocurrió**, porque descartaría filas válidas por motivos que solo le importan al modelo.
- Usar el filtro laxo para todo → el entrenamiento fallaría o exigiría imputación silenciosa.

Con este dataset concreto la diferencia numérica para el modelo de daños resulta ser **cero** (3 958 en ambos casos, porque `Mes`, `Año`, `Clasificación`, `Tipo` y `Estado` no tienen nulos tras la limpieza, y el merge de población es completo). Para el modelo de población la diferencia son **35 filas**. Que hoy la brecha sea pequeña no invalida la separación: **es una salvaguarda de diseño**, y basta con que una futura versión de `Base.xlsx` traiga meses faltantes para que la distinción se vuelva material. Colapsarla sería introducir un acoplamiento entre dos requisitos que no tienen por qué coincidir.

Perfil estadístico del objetivo

Estas cifras son el argumento central del capítulo 6, así que conviene tenerlas a mano.

Total de daños (millones de pesos) — n = 3 958

Estadístico	Valor
Media	168.78
Desviación estándar	1 880.10
Mínimo	0.00
Percentil 10	0.00
Percentil 25	0.00
Mediana	0.16
Percentil 75	5.45
Percentil 90	114.18
Percentil 95	425.03
Percentil 99	2 399.02
Máximo	84 207.02
Ceros	1 065 (26.9 %)
Asimetría (skew)	29.61
Curtosis	1 131.51
Asimetría tras log1p	1.78

Léase la tercera columna con atención: **la media (168.78) es más de mil veces la mediana (0.16)**. La desviación estándar es once veces la media. Una asimetría de 29.6 y una curtosis de 1 131 describen una distribución que no se parece en nada a una campana: es una masa enorme de eventos casi sin costo, con una cola que se extiende cuatro órdenes de magnitud hasta un único evento de 84 207 millones.

Concentración verificada:

- El **1 % de eventos más costosos** (40 eventos) concentra el **66.7 %** del daño total.
- Los **10 eventos más costosos** concentran el **45.0 %**.

Población afectada — n = 3 923

Estadístico	Valor
Media	15 013.4
Desviación estándar	141 975.0
Mediana	85
Percentil 90	10 045.4
Percentil 99	275 461.6
Máximo	4 050 452
Ceros	466 (11.9 %)
Asimetría	19.46
Asimetría tras log1p	0.41

El patrón se repite: media 15 013 contra mediana 85.

Distribución de las variables categóricas

Clasificación del fenómeno (en el conjunto de 3 958 filas del dashboard):

Clasificación	Eventos	%
Hidrometeorológico	3 656	92.4 %
Geológico	302	7.6 %

Tipo de fenómeno (sobre 3 967 filas limpias):

Tipo	Eventos	Clasificación
Lluvias e Inundaciones	2 441	Hidrometeorológico
Frío Extremo	412	Hidrometeorológico
Calor Extremo	292	Hidrometeorológico
Movimientos de Masa	241	Geológico
Ciclones	241	Hidrometeorológico
Sequía	155	Hidrometeorológico
Viento Extremo	124	Hidrometeorológico
Sismos	58	Geológico
Actividad Volcánica	3	Geológico

Este desbalance es una limitación de primer orden. **"Lluvias e Inundaciones" absorbe el 61.5 % de los registros**, mientras que "Actividad Volcánica" tiene tres filas en 24 años. Cualquier estimación del sistema para actividad volcánica descansa sobre tres observaciones: no es un modelo, es una anécdota. El capítulo 11 desarrolla la implicación.

Cobertura temporal

Eventos registrados por año:

Año	n	Daño total (MDP)	Año	n	Daño total (MDP)
2000	34	1 549.6	2012	188	16 818.2
2001	13	2 631.2	2013	168	57 305.7
2002	152	10 954.2	2014	173	30 306.5
2003	197	5 558.0	2015	179	17 442.8
2004	187	714.8	2016	131	11 992.5
2005	143	45 097.4	2017	97	87 712.3
2006	180	4 374.2	2018	223	14 148.1
2007	175	50 338.8	2019	151	9 224.5
2008	237	13 968.0	2020	161	31 279.0
2009	180	14 112.2	2021	239	14 131.2
2010	185	91 156.3	2022	194	15 404.4
2011	242	34 308.3	2023	138	87 507.5

Dos observaciones defendibles:

1. **2000 y 2001 están claramente subrepresentados** (34 y 13 eventos, frente a una media de ~180 en el resto). No es que hubiera menos desastres: es que la compilación de registros para esos años es más incompleta. Un modelo con **Año** como feature puede aprender ese artefacto de registro como si fuera señal.
2. **La serie de daño no tiene tendencia limpia**, la dominan picos: 2010 (91 156), 2017 (87 712), 2023 (87 508), 2013 (57 306), 2007 (50 339). Son años de eventos catastróficos singulares —el sismo de septiembre de 2017 en la Ciudad de México, el huracán Otis en Guerrero en 2023—, no de un incremento gradual.

Los diez eventos más costosos de la base

#	Año	Estado	Tipo	Daño (MDP)
1	2023	Guerrero	Ciclones	84 207.02
2	2017	Ciudad de México	Sismos	43 996.12
3	2007	Tabasco	Lluvias e Inundaciones	31 871.26
4	2010	Veracruz	Ciclones	24 679.80
5	2014	Baja California Sur	Ciclones	24 133.17
6	2013	Guerrero	Ciclones	23 441.40
7	2010	Nuevo León	Ciclones	21 500.86
8	2005	Quintana Roo	Ciclones	18 258.00
9	2005	Chiapas	Ciclones	15 031.50
10	2020	Tabasco	Lluvias e Inundaciones	13 580.56

Siete de los diez son ciclones. Es coherente con la física del riesgo en México y confirma que el modelo tiene un patrón real que aprender, aunque de muy pocos ejemplos extremos.

Los diez estados con mayor daño acumulado

#	Estado	Daño acumulado (MDP)
1	Guerrero	118 142.21
2	Veracruz	77 116.80
3	Tabasco	71 916.93
4	Chiapas	56 474.95
5	Ciudad de México	45 371.22
6	Oaxaca	43 569.75
7	Baja California Sur	35 072.42
8	Nuevo León	27 355.46
9	Quintana Roo	26 018.31
10	Sinaloa	14 457.57

CAPÍTULO 5

El modelo de aprendizaje automático

Este capítulo es el núcleo del manual. Cada sección plantea una decisión, da la alternativa que se descartó y explica con evidencia por qué.

Qué se está modelando exactamente

Antes de discutir algoritmos conviene fijar la pregunta, porque de ella se derivan todos los límites del sistema.

El modelo aprende una función:

$$f(\text{clasificación, tipo de fenómeno, estado, año, mes}) \rightarrow \text{impacto esperado}$$

Nótese lo que **no** aparece en la entrada: ninguna medida de la intensidad del evento. No hay categoría Saffir-Simpson, ni magnitud en escala de momento, ni milímetros de precipitación. El modelo por tanto no estima "el impacto de *este* huracán": estima **el impacto promedio, en escala logarítmica, de la clase de eventos que comparten esas cinco características**.

Formulado como pregunta que el sistema sí responde:

"Dado que ocurre un ciclón en Guerrero en octubre, ¿cuánto impacto cabe esperar según lo que ocurrió históricamente en situaciones equivalentes?"

Y como pregunta que **no** responde:

"¿Ocurrirá un ciclón en Guerrero en octubre?" — eso es meteorología, no está en el alcance.

Por qué `HistGradientBoostingRegressor`

`HistGradientBoostingRegressor` es la implementación de scikit-learn de *gradient boosting* con discretización por histogramas: construye una secuencia de árboles de decisión pequeños donde cada árbol corrige el error residual que dejó la suma de los anteriores. Es la variante de scikit-learn inspirada en LightGBM.

Se eligió frente a tres alternativas naturales.

Frente a una regresión lineal

Una regresión lineal (o Ridge, o Lasso) asume que el objetivo es una suma ponderada de las entradas. Aquí eso es falso por dos motivos:

1. **Las interacciones son el fenómeno.** El daño de "ciclón" no es un número que se suma al número de "Guerrero". Un ciclón en Guerrero en octubre es catastrófico; el mismo ciclón en Zacatecas en marzo es irrelevante. El efecto del tipo de fenómeno **depende** del estado y del mes. Un modelo lineal solo captura eso si se le construyen a mano los términos de interacción, que con $2 \times 9 \times 32$ combinaciones categóricas serían cientos de columnas nuevas. Un árbol las descubre solo, porque cada rama es literalmente una interacción condicional.
2. **La relación no es monótona ni suave.** No existe un "coeficiente del mes": el daño sube en septiembre y octubre, cae en marzo, repunta ligeramente en enero por el frío extremo. Un árbol representa esa forma sin esfuerzo; una recta no.

Evidencia empírica reproducida para este manual. Se entrenó una Ridge sobre exactamente el mismo preprocesamiento, el mismo `log1p` y el mismo split temporal:

Modelo	R ² en escala log (prueba ≥ 2020)	MAE
Ridge + <code>log1p</code>	-0.0130	204.35
<code>HistGradientBoostingRegressor</code> + <code>log1p</code>	0.1661	209.09

Un R² negativo significa que la regresión lineal predice **peor que responder siempre la media del conjunto de prueba**. El árbol impulsado extrae señal donde la recta no encuentra ninguna.

Frente a una red neuronal

Una red neuronal es el instrumento equivocado para este problema por tres razones concretas:

1. **Tamaño de muestra.** 3 958 filas con 47 columnas después del one-hot. Las redes profundas necesitan órdenes de magnitud más datos para no memorizar el conjunto de entrenamiento. Con 732 filas de prueba, la varianza de cualquier métrica ya es apreciable; añadir un modelo con decenas de miles de parámetros agravaría el sobreajuste sin ganancia.
2. **Naturaleza tabular y mixta.** La literatura empírica sobre datos tabulares —donde conviven variables categóricas de alta cardinalidad y numéricas de escalas dispares— muestra de forma consistente que los ensambles de árboles igualan o superan a las redes con una fracción del esfuerzo de ajuste. Las redes brillan donde hay estructura espacial o secuencial (imagen, audio, texto); aquí no la hay.
3. **Costo de operación y defensa.** Una red exige normalización de entradas, elección de arquitectura, tasa de aprendizaje, regularización, criterio de paro y varias corridas para estabilizar. El `HistGradientBoostingRegressor` funciona con sus valores por omisión y es explicable en una frase. Para un proyecto que debe defenderse, la explicabilidad tiene valor propio.

Frente a un `RandomForestRegressor` o `ExtraTreesRegressor`

Es la alternativa más cercana, y de hecho **la versión anterior de este proyecto usaba `ExtraTreesRegressor`** (visible en el historial de Git, commit `0aa40c7`). El cambio a boosting llegó junto con el rediseño ex-ante. Las ventajas del boosting aquí:

- **Manejo nativo de valores faltantes.** `HistGradientBoostingRegressor` decide en cada nodo hacia qué lado enviar los `NaN`; los bosques aleatorios de scikit-learn no aceptan `NaN`. Aunque el pipeline actual elimina las filas incompletas, la propiedad da margen para futuras versiones del dataset.
- **Velocidad.** La discretización en histogramas (256 cubetas por variable numérica) hace que el entrenamiento sea de segundos.
- **Tamaño del artefacto.** El modelo `ExtraTrees` anterior pesaba **45.7 MB**; los dos modelos actuales pesan **370 KB cada uno**, una reducción de más de 100×. En un despliegue con límite de tamaño de bundle esto es la diferencia entre poder desplegar y no poder.

Hiperparámetros

El modelo se instancia sin ajuste:

```
model = HistGradientBoostingRegressor(random_state=42)
```

Los valores efectivos, verificados sobre el artefacto serializado:

Hiperparámetro	Valor	Efecto
<code>loss</code>	<code>squared_error</code>	minimiza el error cuadrático (en escala log)
<code>learning_rate</code>	0.1	contribución de cada árbol al total
<code>max_iter</code>	100	número máximo de árboles
<code>n_iter_</code> (real)	100	se usaron los 100; no hubo paro temprano
<code>max_leaf_nodes</code>	31	profundidad efectiva de cada árbol
<code>min_samples_leaf</code>	20	mínimo de filas por hoja; frena el sobreajuste
<code>l2_regularization</code>	0.0	sin penalización L2
<code>early_stopping</code>	<code>auto</code>	se activaría con más de 10 000 filas; aquí no se activa
<code>random_state</code>	42	reproducibilidad exacta entre corridas

Dos comentarios honestos sobre esto:

- `random_state=42` es lo que hace el sistema reproducible. Sin él, dos entrenamientos consecutivos producirían modelos ligeramente distintos y las métricas de `metadata.json` no serían verificables.
- **No hay búsqueda de hiperparámetros.** No se ejecutó `GridSearchCV` ni `RandomizedSearchCV`. El capítulo 11 lo registra como área de oportunidad, y también aclara por qué el margen de mejora por esa vía probablemente sea pequeño comparado con el que dan mejores variables de entrada.

Por qué dos modelos independientes con un preprocesador compartido

El sistema entrena **dos regresores separados**, uno por objetivo:

```

model_danos, prep_danos, n_danos = fit_final(df, TARGET_DANOS)
model_pobl, _, n_pobl = fit_final(df, TARGET_POBL)
preprocessor = prep_danos # ambos comparten features -> un preprocesador basta

```

Por qué no un modelo multi-salida

`scikit-learn` permite `MultiOutputRegressor` o pasar una `y` de dos columnas. Se descartó por tres razones:

1. **Los objetivos tienen distinta disponibilidad.** El daño está reportado en 3 958 filas; la población afectada en 3 923. Un modelo multi-salida obligaría a usar la intersección (3 923) o a imputar, desperdiciando 35 filas del modelo de daños sin ninguna ganancia.
2. **Los objetivos tienen distinta distribución.** Daño: mediana 0.16, máximo 84 207, 26.9 % de ceros. Población: mediana 85, máximo 4 050 452, 11.9 % de ceros. Un solo modelo optimizando una pérdida conjunta reparte capacidad entre dos superficies muy distintas.
3. **Independencia operativa.** Con dos artefactos, se puede reentrenar, reemplazar o retirar uno sin tocar el otro. De hecho el código ya lo contempla: `predict()` incluye la población en la respuesta solo si `model_pobl is not None`. El sistema degrada con elegancia.

Por qué no un modelo en cascada

Una alternativa considerada y descartada en la especificación de diseño era predecir primero la población afectada y usar esa predicción como entrada del modelo de daños. Se rechazó porque **encadenar propaga el error**: el modelo de población es el más débil de los dos ($R^2 \log 0.103$), así que alimentar sus salidas al modelo de daños inyectaría ruido en lugar de información.

Por qué un solo preprocesador

Ambos modelos consumen **exactamente el mismo vector de entrada**. Ajustar dos `ColumnTransformer` idénticos sería duplicar 4 KB de artefacto y, peor, crear la posibilidad de que diverjan —por ejemplo, si un modelo se reentrenara con un dataset que contiene un tipo de fenómeno nuevo y el otro no, los vectores dejarían de ser comparables y `predict()` no podría reutilizar una sola transformación. Un preprocesador compartido convierte esa clase de error en imposible por construcción.

Qué hace el preprocesador

```

ColumnTransformer(transformers=[
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), CAT_FEATURES),
    ("num", "passthrough", NUM_FEATURES),
])

```

Dimensiones verificadas sobre el artefacto real:

Bloque	Columnas de entrada	Columnas de salida
One-hot <code>Clasificación del fenómeno</code>	1	2
One-hot <code>Tipo de fenómeno</code>	1	9
One-hot <code>Estado</code>	1	32
Passthrough <code>Año</code> , <code>mes_sin</code> , <code>mes_cos</code> , <code>Población estatal</code>	4	4
Total	7	47

Por qué one-hot y no ordinal. Codificar los estados como 0...31 le diría al árbol que Baja California (1) está "entre" Aguascalientes (0) y Baja California Sur (2), y que Zacatecas (31) es el "mayor". Ese orden no existe: son categorías nominales. El one-hot elimina el orden espurio a cambio de más columnas, un costo trivial con solo 32 categorías.

Por qué no se escalan las numéricas. Los árboles parten por umbrales (`Año ≤ 2016.5`); la escala de la variable no altera qué particiones son posibles. Escalar sería trabajo inútil, y además volvería los umbrales ilegibles al inspeccionar el modelo.

`handle_unknown="ignore"` : la razón y su costo. Si en producción llega una categoría que el preprocesador no vio al ajustarse, en vez de lanzar excepción codifica ese bloque como todo ceros. Evita que la API se caiga. Pero tiene un efecto que hay que conocer y que se verificó experimentalmente: enviar un tipo de fenómeno inexistente, `"Tsunami"`, **no produce error**, produce una predicción de 1.71 MDP; enviar un estado inexistente, `"Narnia"`, produce 126.37 MDP usando la mediana nacional de población como respaldo. El sistema responde con seguridad a preguntas sin sentido. El capítulo 11 propone la corrección.

Por qué la transformación logarítmica del objetivo

Este es probablemente el punto más importante del modelo, y el más fácil de defender porque hay evidencia numérica directa.

El mecanismo

```
model.fit(X, np.log1p(y))           # entrenar sobre log(1 + y)
pred = np.maximum(np.expm1(model.predict(X)), 0) # invertir con exp(x) - 1
```

`log1p(y) = ln(1 + y)` y `expm1(x) = ex - 1` son funciones inversas. Se usa la variante "1p" en lugar de `log(y)` porque **el 26.9 % de los daños son exactamente cero** y `log(0)` es $-\infty$. Con `log1p(0) = 0` los ceros quedan representados sin problema.

Son un par: entrenar con `log1p` y **no** invertir con `expm1` devolvería el logaritmo del daño como si fueran millones de pesos. En el código, las dos mitades del par están en archivos distintos —

`train_model_simple.py` aplica `log1p`, `main.py` aplica `expm1` — lo que las convierte en un acoplamiento implícito que hay que mantener sincronizado a mano.

Por qué es necesaria

Recuérdense las cifras del capítulo 5: asimetría **29.61**, curtosis **1 131.5**, media 168.78 contra mediana 0.16, máximo 84 207.

La pérdida que el modelo minimiza es el **error cuadrático**. Con datos así, el cuadrado del error del evento de 84 207 MDP domina la suma: si el modelo predice 1 000 para ese evento, aporta un residuo al cuadrado de aproximadamente 6.9×10^9 . Los 1 065 eventos de daño cero, aunque el modelo los prediga con un error de 10 MDP cada uno, aportan en conjunto $\sim 1.1 \times 10^5$ — **cinco órdenes de magnitud menos**. El optimizador, racionalmente, dedica toda su capacidad a los diez eventos catastróficos e ignora los 3 948 restantes.

Aplicar `log1p` comprime la escala: 84 207 se convierte en 11.34; 0.16 se convierte en 0.15. La asimetría cae de **29.61 a 1.78** (y en el objetivo de población, de **19.46 a 0.41**, casi simétrica). Con eso, un error de "un factor de 2" pesa lo mismo en un evento grande que en uno pequeño, que es exactamente el comportamiento deseado: en riesgo de desastres importa el **orden de magnitud**, no el peso exacto.

Qué pasa sin ella: experimento verificable

Se entrenó el mismo modelo, con el mismo preprocesador y el mismo split temporal, directamente sobre `y` sin transformar:

Configuración	R ² en escala log	MAE	MedAE
<code>HistGradientBoostingRegressor</code> + <code>log1p</code>	0.1661	209.09	1.69
<code>HistGradientBoostingRegressor</code> sin <code>log1p</code>	-1.8610	344.68	20.79

Sin la transformación el R² logarítmico se hunde a **-1.86** —muy por debajo de predecir siempre una constante— el MAE empeora un 65 % y el error mediano se multiplica por **12**. El modelo sin `log1p` es peor que inútil para el caso típico: se dedica a perseguir catástrofes y aplasta los miles de eventos ordinarios.

El precio de la transformación: sesgo de retransformación

Ahora la parte honesta, y es un punto que conviene tener preparado porque un sinodal atento puede señalarlo.

Entrenar con error cuadrático en escala logarítmica hace que el modelo estime la **media condicional del logaritmo**. Al aplicar `expm1`, lo que se recupera es aproximadamente la **mediana geométrica**, no la media aritmética. Por la desigualdad de Jensen, $\exp(E[\log y]) \leq E[y]$, y la brecha crece con la varianza de la distribución — que aquí es enorme.

Medición directa sobre el conjunto de prueba (años ≥ 2020 , 732 eventos):

Magnitud	Valor
Suma real del daño	148 322.2 MDP
Suma predicha por el modelo	13 131.7 MDP
Cobertura	8.9 %

El modelo subestima el agregado en más de un orden de magnitud. Es la consecuencia esperada de optimizar en escala logarítmica, y tiene una implicación de alcance categórica: **estas predicciones no deben sumarse para presupuestar un total nacional.** Sirven para comparar escenarios entre sí ("¿cuál de estos dos estados espera más impacto?"), no para agregar.

Este mismo efecto explica por qué las predicciones de población salen tan bajas. Con la mediana del objetivo en 85 personas, el modelo devuelve valores como 161 o 400 personas para escenarios que intuitivamente afectan a miles. No es un bug de código: es lo que un modelo entrenado en escala logarítmica devuelve sobre una distribución con esa mediana. El capítulo 11 propone la corrección (factor de suavizado de Duan, o un modelo en dos etapas).

Por qué las variables cíclicas del mes

El problema

El mes es un entero de 1 a 12, pero **es circular**: diciembre y enero son meses consecutivos. Si se entrega al modelo como número:

- La distancia entre diciembre (12) y enero (1) es **11**.
- La distancia entre enero (1) y febrero (2) es **1**.

Numéricamente, diciembre y enero son los dos meses más lejanos del calendario, cuando en realidad son adyacentes. Un árbol de decisión partiría por umbrales como `Mes ≤ 11.5`, agrupando enero con la primavera y aislando diciembre. Para capturar "la temporada fría abarca diciembre y enero" el árbol necesitaría dos ramas separadas y, por tanto, el doble de datos.

La solución

Se proyecta el mes a un punto sobre una circunferencia:

```
df["mes_sin"] = np.sin(2 * np.pi * df["Mes"] / 12)
df["mes_cos"] = np.cos(2 * np.pi * df["Mes"] / 12)
```

Cada mes queda representado por un par de coordenadas:

Mes	mes_sin	mes_cos
1 — enero	0.500	0.866
3 — marzo	1.000	0.000
6 — junio	0.000	-1.000
9 — septiembre	-1.000	0.000
12 — diciembre	0.000	1.000

La distancia euclidiana entre diciembre (0, 1) y enero (0.5, 0.866) es 0.518, la misma que entre cualquier otro par de meses consecutivos. **La circularidad quedó codificada en la geometría del espacio de entrada**, y el modelo no necesita aprenderla.

Por qué dos variables y no una

Con solo el seno, marzo ($\sin = 1.0$) y... bueno, marzo sería único, pero febrero ($\sin = 0.866$) y abril ($\sin = 0.866$) serían **indistinguibles**. Se necesitan dos coordenadas para ubicar un punto sin ambigüedad en una circunferencia. Seno y coseno juntos identifican cada mes de manera única.

Por qué no one-hot del mes

Sería la otra alternativa razonable: 12 columnas binarias. Se descartó porque añadiría 12 columnas al vector (de 47 a 58, un 25 % más) **y perdería la información de vecindad**: con one-hot, el modelo trata "septiembre" y "octubre" como categorías sin relación, y no puede generalizar de una a otra si tiene pocos datos de octubre. La codificación cíclica usa **dos** columnas y conserva la proximidad.

Verificación de que funciona

Se consultó al modelo servido con un mismo escenario —ciclón en Guerrero, 2026— variando únicamente el mes:

Mes	Daño estimado (MDP)	Mes	Daño estimado (MDP)
Enero	87.72	Julio	90.75
Febrero	80.04	Agosto	131.66
Marzo	59.79	Septiembre	397.99
Abril	78.83	Octubre	639.94
Mayo	72.79	Noviembre	110.08
Junio	8.95	Diciembre	92.54

El modelo aprendió la **temporada de ciclones del Pacífico mexicano**: el pico en septiembre y octubre, con un descenso hacia el invierno y la primavera. Nadie se lo dijo; lo extrajo de los datos gracias a que la representación del mes se lo permitía.

Fuga de datos: el punto fuerte del proyecto

Qué es una fuga de datos

Se produce **fuga de información** (*data leakage*) cuando el modelo recibe, entre sus variables de entrada, información que en el momento real de uso todavía no existiría. El síntoma es un modelo con métricas excelentes en evaluación y comportamiento inútil en producción, porque en producción esa variable no está disponible — o, peor, porque solo se conoce **después** de conocer la respuesta que el modelo debía predecir.

La fuga que tenía la versión anterior

La primera versión de este proyecto (commit `0aa40c7`, `ExtraTreesRegressor` con siete entradas) incluía entre sus features las columnas `Impacto humano` y `Daños a infraestructura`.

Ambas son escalas ordinales de severidad de 0 a 5 —verificado: los valores presentes son exactamente `{0, 1, 2, 3, 4, 5}`— que el analista original asignó a cada evento **a partir de los daños reportados**. Es decir: son **componentes del propio objetivo**, codificados como categorías.

Su correlación con el objetivo en escala logarítmica, medida sobre los datos reales:

Columna post-evento	Correlación con <code>log1p(Total de daños)</code>
<code>Impacto humano</code>	0.359
<code>Daños a infraestructura</code>	0.261

Usarlas es circular. Equivale a preguntarle al modelo "*¿cuánto costó el desastre?*" dándole como pista "*fue un desastre de severidad 4 sobre 5*". En el momento en que un usuario de protección civil quiere una estimación —antes de que ocurra el evento, o mientras está ocurriendo— **nadie ha asignado todavía esa severidad**. La variable simplemente no existe.

La decisión y su costo medido

La especificación de diseño (`docs/superpowers/specs/2026-06-13-modelos-ex-ante-design.md`) documenta el diagnóstico y la resolución: eliminar ambas columnas y operar en modo estrictamente ex-ante. El código lo hace explícito desde el docstring:

```
"""...NO usan columnas post-evento (Impacto humano, Daños a infraestructura),
que serían leakage."""
```

Y `PredictRequest` en la API sencillamente ya no las acepta.

El costo es real y hay que reconocerlo. Se reentrenó el modelo incluyendo las dos columnas post-evento, con idéntico preprocesamiento y split temporal:

Configuración	R ² en escala log (prueba ≥ 2020)	MAE
Con columnas post-evento (con fuga)	0.4448	177.70
Sin ellas — el modelo entregado	0.1661	209.09

Es decir: **el proyecto renunció voluntariamente a que su R² casi se triplicara.**

Ese es el argumento central de defensa del trabajo. Un modelo con 0.44 se reporta mejor en una presentación, pero es un modelo que **no puede ejecutarse nunca en el momento en que serviría de algo**, porque una de sus entradas solo se conoce después de la respuesta. Un modelo con 0.17 es más pobre y es **operable**. Preferir el segundo es una decisión metodológica, no un accidente, y está documentada por escrito antes de implementarse.

Cómo se sostiene la ausencia de fuga

Tres mecanismos la garantizan:

1. La lista **FEATURES** es explícita y se declara una sola vez en `train_model_simple.py`. No hay `df.drop(columns=[target])` ni selección por exclusión, que es donde suelen colarse variables sin querer.
2. **PredictRequest** es el contrato de la API: acepta cinco campos y Pydantic rechaza cualquier otro. Es imposible que la API alimente al modelo con algo que el usuario no pueda conocer de antemano.
3. `metadata.json` registra `"mode": "ex-ante"` y la lista de features, y `/health` los expone. El modo del modelo es auditable en caliente.

Split temporal en lugar de aleatorio

Qué se hizo

```
TEST_YEAR_FROM = 2020

def evaluate_temporal(df, target):
    d = df.dropna(subset=FEATURES + [target]).copy()
    train = d[d["Año"] < TEST_YEAR_FROM] # 2000-2019
    test = d[d["Año"] >= TEST_YEAR_FROM] # 2020-2023
```

Conjunto	Años	Filas (modelo de daños)	Filas (modelo de población)
Entrenamiento	2000-2019	3 226	3 191
Prueba	2020-2023	732	732

Aproximadamente **81 % / 19 %**, sin barajar.

Por qué el split aleatorio sería deshonesto aquí

La validación cruzada estándar reparte las filas al azar. Con datos temporales, eso produce dos formas de optimismo indebido:

1. El modelo "ve el futuro". Con un reparto aleatorio, el modelo entrena con eventos de 2022 y se evalúa con eventos de 2015. Pero el uso real es el contrario: se entrena con lo pasado y se aplica a lo que aún no ocurre. Una métrica obtenida mirando hacia atrás no dice nada sobre el desempeño hacia adelante.

2. Contaminación por eventos hermanos. Este es el problema más grave con esta base concreta. Un huracán que golpea varios estados genera **varias filas casi idénticas** —mismo año, mismo mes, mismo tipo, magnitudes de daño similares—. Un reparto aleatorio pone algunas de esas filas en entrenamiento y otras en prueba. El modelo memoriza la fila de entrenamiento y "acierta" su gemela en prueba. Eso no es generalización: es recuperación de memoria.

Cuánto infla, medido

Se ejecutó el mismo modelo con un split aleatorio 80/20 (`random_state=42`):

Esquema de validación	R ² en escala log	MAE	MedAE
Split aleatorio 80/20	0.3821	173.36	1.33
Split temporal (< 2020 / ≥ 2020)	0.1661	209.09	1.69

El split aleatorio reporta un R² **2.3 veces mayor**. Ambas cifras salen del mismo modelo y los mismos datos; lo único que cambia es cómo se decidió qué filas ocultar. **La diferencia entre 0.38 y 0.17 es exactamente la magnitud del autoengaño que evita el split temporal.**

Reportar 0.38 no sería mentir sobre un número: sería responder una pregunta distinta de la que el sistema debe responder.

El detalle del reentrenamiento final

Hay un matiz importante:

```
def fit_final(df, target):
    """Reentrena con TODOS los años para el modelo que se sirve."""
    d = df.dropna(subset=FEATURES + [target]).copy()
    model.fit(pre_fit_transform(d[FEATURES]), np.log1p(d[target].values))
```

El modelo que se guarda y se sirve **usa las 3 958 filas, incluidos 2020–2023**. La separación temporal existe únicamente para *medir*.

Esto es práctica correcta y estándar: una vez que la validación estableció qué tan bien generaliza el procedimiento, se reajusta con todos los datos para no desperdiciar los cuatro años más recientes —que además son los más relevantes para predecir el futuro inmediato. Pero implica una precisión que hay que hacer al presentarlo:

Las métricas de `metadata.json` **no** son las métricas del modelo que está en producción. Son las métricas de un modelo entrenado con el mismo procedimiento sobre menos datos. Es la estimación honesta disponible del desempeño esperado del modelo servido, no una medición directa de él.

Interpretación honesta de las métricas

Las cifras oficiales

De `backend/app/artifacts/metadata.json` :

Modelo	Objetivo	R ² log	MAE	MedAE	n entrena	n prueba
A — daños	Total de daños (millones de pesos)	0.1661	209.09 MDP	1.6924 MDP	3 226	732
B — población	Población afectada	0.1033	4 380.21 personas	49.6169 personas	3 191	732

Qué significa cada una

R²log = 0.1661. El R² se calcula sobre `log1p(real)` contra `log1p(predicho)`, no sobre los pesos directamente. Significa que el modelo explica el **16.6 % de la varianza del logaritmo del daño**. En escala lineal, con un solo evento aportando el 12.6 % del daño total de la base, el R² sería tan volátil que dependería de si ese evento cayó en entrenamiento o en prueba; en escala logarítmica la métrica es estable e interpretable.

Contexto necesario para juzgar 0.166: **la especificación de diseño esperaba 0.37–0.40** para el modelo A. El resultado quedó por debajo de lo previsto. Esa brecha entre expectativa documentada y resultado obtenido está en el repositorio y es un dato del proyecto, no algo que ocultar.

MAE = 209.09 MDP y MedAE = 1.69 MDP. La distancia entre estas dos cifras —un factor de 124— es la firma inequívoca de una distribución con cola pesada. El error *mediano* es de 1.69 millones de pesos: en la mitad de los casos el modelo se equivoca por menos de eso. El error *promedio* es de 209 millones porque unos pocos eventos catastróficos, que el modelo no puede anticipar sin datos de intensidad, arrastran la media.

La distribución completa del error absoluto sobre las 732 filas de prueba:

Percentil del error absoluto	Valor (MDP)
25	0.45
50 (mediana)	1.69
75	6.43
90	120.73
95	440.62
99	1 820.10

Lectura defendible: el modelo es razonablemente competente en el régimen ordinario —tres de cada cuatro predicciones se equivocan por menos de 6.43 millones de pesos— y **falla en los eventos extremos, que son precisamente los que más importan**. Esta es la limitación honesta del sistema, y no se resuelve con un mejor algoritmo: se resuelve con variables de intensidad del evento, que hoy no están en los datos.

Contra qué comparar: las líneas base

Un R^2 de 0.166 no significa nada en el vacío. Se calcularon dos referencias triviales sobre el mismo conjunto de prueba:

Modelo	$R^2\log$	MAE	MedAE
Predecir siempre la media del entrenamiento	-4.7769	331.95	161.08
Predecir siempre la mediana del entrenamiento	-0.1446	202.65	0.23
Regresión Ridge + log1p	-0.0130	204.35	4.61
HGB + log1p (el sistema)	0.1661	209.09	1.69
HGB + log1p , split aleatorio (inflado)	0.3821	173.36	1.33
HGB con fuga de columnas post-evento	0.4448	177.70	—

Dos conclusiones se sostienen con esta tabla:

- 1. El modelo aprende algo real.** Es el único de los tres candidatos legítimos con $R^2\log$ positivo; la media, la mediana y la regresión lineal quedan todos en negativo.
- 2. La ventaja no es enorme y hay que decirlo.** En MAE, la mediana constante (202.65) es *ligeramente mejor* que el modelo (209.09). El modelo gana claramente en $R^2\log$ y en estructura —discrimina entre escenarios, la mediana constante no—, pero el margen sobre una regla trivial es estrecho. Sostener lo contrario sería vender algo que los datos no respaldan.

Nota sobre el modelo B. $R^2\log = 0.1033$ es aún más bajo. La especificación esperaba ≈ 0.31 . Además, ese objetivo arrastra un problema de calidad de datos propio: 44 filas con **"SD"** y 466 con valor cero, muchas de las cuales probablemente son "no se contabilizó" registrado como 0 en lugar de como dato faltante.

Un cero espurio es peor que un hueco: el modelo lo aprende como observación válida.

CAPÍTULO 6

La API

Marco y arranque

La API está construida con **FastAPI 0.115.0** sobre **Uvicorn 0.30.6**, con **Python 3.11.9** fijado en `backend/runtime.txt`. FastAPI aporta tres cosas que el proyecto usa activamente: validación automática de la petición mediante Pydantic, generación automática de documentación interactiva en `/docs`, y serialización de respuestas a JSON.

La carga de artefactos ocurre **en el cuerpo del módulo**, no dentro de un manejador de arranque:

```
if MODEL_DANOS_PATH.exists() and PREP_PATH.exists():
    model_danos = joblib.load(MODEL_DANOS_PATH)
    preprocessor = joblib.load(PREP_PATH)
    if MODEL_POBL_PATH.exists():
        model_pobl = joblib.load(MODEL_POBL_PATH)
    ...
```

La ventaja es que los modelos se deserializan una sola vez, al importar el módulo, y quedan en memoria para todas las peticiones. Cargar 370 KB de modelo en cada `/predict` sería inaceptable.

El costo —registrado como limitación en el capítulo 11— es que si los archivos no existen, las variables globales quedan en `None` y el error se manifiesta más tarde, en forma de una respuesta HTTP 200 con una clave `"error"` dentro. No hay `try/except`, ni registro estructurado, ni código de estado que un monitor externo pueda detectar.

Dependencias fijadas

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
pandas==2.2.2
numpy==2.0.1
scikit-learn==1.5.1
joblib==1.4.2
openpyxl==3.1.5
pydantic==2.9.2
```

Todas las versiones están **clavadas con** `==`, no con `>=`. Es deliberado y hay un commit específico al respecto (`b923c9d`, *"Fijar numpy==2.0.1 para que coincida con los artifacts pickled"*).

La razón es que `joblib` serializa objetos de Python mediante *pickle*, que guarda referencias a las clases de la biblioteca que los creó. Si el servidor tuviera `scikit-learn` 1.6 y los artefactos se hubieran generado con 1.5.1, la deserialización puede fallar o —peor— tener éxito con comportamiento distinto.

Lo mismo aplica a `numpy`, cuyos arreglos internos viajan dentro del pickle. La fijación de versiones es lo que hace que el artefacto sea portable entre la máquina de entrenamiento y el servidor.

GET / — sonda mínima

```
{"message": "API activa. Ve a /docs para probar endpoints."}
```

GET /health — diagnóstico

Sin parámetros. Responde:

```
{
  "status": "ok",
  "model_danos_loaded": true,
  "model_poblacion_loaded": true,
  "preprocessor_loaded": true,
  "poblacion_loaded": true,
  "stats_loaded": true,
  "mode": "ex-ante",
  "features": ["Clasificación del fenómeno", "Tipo de fenómeno", "Estado",
              "Año", "mes_sin", "mes_cos", "Población estatal"],
  "models": { "danos": {...}, "poblacion": {...} }
}
```

Es más que un *ping*: reporta **qué artefactos se cargaron efectivamente** y devuelve el contenido de `metadata.json`, incluidas las métricas temporales. Un operador puede verificar de un vistazo si el servidor está sirviendo el modelo correcto y en qué modo.

Limitación: `"status"` es la cadena literal `"ok"` siempre, y el código HTTP es 200 aunque todos los artefactos estén en `false`. Un sistema de monitoreo que solo mire el código de estado no detectaría un despliegue sin modelos.

POST /predict — el endpoint principal

Contrato de entrada

```
class PredictRequest(BaseModel):
    Año: int
    Mes: int
    Clasificación_del_fenómeno: str
    Tipo_de_fenómeno: str
    Estado: str
```

Ejemplo de cuerpo:

```
{
  "Año": 2026,
  "Mes": 10,
  "Clasificación_del_fenómeno": "Hidrometeorológico",
  "Tipo_de_fenómeno": "Ciclones",
  "Estado": "Guerrero"
}
```

El remapeo de guion bajo a espacio

Los identificadores de Python no admiten espacios: no se puede escribir `Clasificación del fenómeno: str` como atributo de una clase. Pero el preprocesador fue ajustado sobre columnas **con** espacios. La traducción ocurre al construir la fila:

```
row = {
  "Clasificación del fenómeno": d["Clasificación_del_fenómeno"],
  "Tipo de fenómeno":          d["Tipo_de_fenómeno"],
  "Estado":                     estado,
  "Año":                        d["Año"],
  "mes_sin":                    np.sin(2 * np.pi * mes / 12),
  "mes_cos":                    np.cos(2 * np.pi * mes / 12),
  "Población estatal":          _poblacion_estatal(estado, d["Año"]),
}
X = preprocessor.transform(pd.DataFrame([row])[FEATURES])
```

El `[FEATURES]` final no es adorno: **fuerza el orden exacto de columnas** que el `ColumnTransformer` espera. Un diccionario de Python conserva el orden de inserción, pero depender de eso sería frágil; la indexación explícita lo hace determinista.

Obsérvese también que **la API deriva `mes_sin / mes_cos` y consulta la población**: el cliente envía cinco campos y el modelo recibe siete. Esas dos transformaciones son responsabilidad del servidor, lo que impide que un frontend mal implementado calcule mal las variables cíclicas.

Normalización del estado

```
STATE_ALIASES = {"CDMX": "Ciudad de México", "Estado de México": "México"}
...
estado = d["Estado"].split(",")[0].strip()
estado = STATE_ALIASES.get(estado, estado)
```

Replica en runtime la normalización que `clean_base()` hizo en entrenamiento, y es imprescindible: el modelo aprendió la categoría `"México"`, no `"Estado de México"`. Si el frontend enviara la etiqueta larga sin traducir, el one-hot la ignoraría (`handle_unknown="ignore"`) y ese estado quedaría codificado como todo ceros — una predicción silenciosamente degradada.

Detalle a notar: aquí solo se parte por coma, mientras que en entrenamiento se parte por coma o por "y ". Es una asimetría menor —el frontend usa un `<select>` con valores individuales, así que nunca envía listas— pero es una divergencia real entre las dos rutas de normalización.

Búsqueda de población estatal

```
def _poblacion_estatal(estado: str, año: int) -> float:
    if poblacion is None:
        return 0.0
    yr = min(max(año, poblacion["year_min"]), poblacion["year_max"])
    val = poblacion["by_state_year"].get((estado, yr))
    if val is None:
        return poblacion["national_median"]
    return val
```

Tres comportamientos de respaldo, en cascada:

1. Si el artefacto de población no cargó, devuelve `0.0`.
2. Si el año está fuera del rango 2000–2023, lo **acota** a los extremos. Una petición para 2026 usa la población de 2023 (verificado: 3 540 685 para Guerrero).
3. Si el estado no está en el diccionario, usa la **mediana nacional** — 3 054 892 habitantes.

Contrato de salida

```
{
  "prediction": {
    "Total de daños (millones de pesos)": 639.94,
    "Población afectada": 400.0
  }
}
```

La inversión de la transformación y el truncamiento:

```
danos = float(max(np.expm1(model_danos.predict(X))[0], 0))
```

El `max(..., 0)` existe porque `expm1` de un valor negativo devuelve un número entre -1 y 0 . Un daño negativo no tiene sentido físico, así que se trunca.

La clave `"Población afectada"` solo aparece si `model_pobl` cargó correctamente. El frontend lo maneja con `pred?.["Población afectada"] || 0`.

Comportamiento con entradas fuera de dominio — verificado

Se probó directamente contra el modelo servido:

Entrada	Comportamiento observado
Mes: 99	Acepta. Devuelve 389.49 MDP y 13 649 personas
Mes: -5 , Año: 1800	Acepta. Devuelve 35.69 MDP y 445 personas
Tipo_de_fenómeno: "Tsunami"	Acepta. Devuelve 1.71 MDP
Estado: "Narnia"	Acepta. Devuelve 126.37 MDP
Año: 2050	Acepta. Idéntico a 2023 (397.99 MDP)

Pydantic valida que los campos existan y sean del tipo correcto, pero **no valida rangos ni pertenencia a un conjunto**. Las cuatro primeras filas de esta tabla son respuestas con apariencia de autoridad a preguntas sin sentido. El capítulo 11 propone la corrección concreta.

La quinta fila merece un comentario aparte porque **no es un defecto**: los árboles de decisión son funciones constantes a trozos y no extrapolan. Más allá del último umbral aprendido para `Año`, la predicción se congela. Combinado con el acotamiento del año en la búsqueda de población, cualquier año ≥ 2023 produce el mismo resultado. Es preferible a que un modelo lineal extrapolara una tendencia inventada hasta 2050 — pero conviene saberlo, porque **el formulario propone el año actual (2026) por omisión**, que ya está fuera del rango entrenado.

Los ocho endpoints de estadísticas

Todos operan sobre `stats_df`, el `DataFrame` de $3\,958 \times 31$ cargado desde `data.joblib`. **Ninguno toca el Excel ni los modelos.**

Seis son endpoints de **agregados fijos**, escritos para la primera versión del dashboard. Dos son posteriores y sirven al dashboard interactivo actual.

Endpoint	Parámetros	Agregación	Forma de la respuesta
GET /stats/kpis	—	totales globales	{total_eventos, total_daños, poblacion_afectada, estados_afectados}
GET /stats/evolucion-anual	—	groupby("Año").sum()	[{año, daños}]
GET /stats/top-estados	limit (10)	groupby("Estado").sum() , ordenado	[{estado, daños}]
GET /stats/por-estado	—	groupby("Estado").agg(...)	[{estado, daños, poblacion, total_eventos}]
GET /stats/clasificacion	—	value_counts()	[{label, value, color}]
GET /stats/top-eventos	limit (10)	sort_values descendente	[{año, estado, tipo, daños}]

Valores reales que devuelve `/stats/kpis` con los artefactos actuales:

```
{
  "total_eventos": 3958,
  "total_daños": 668035.68,
  "poblacion_afectada": 58897609,
  "estados_afectados": 32
}
```

Un detalle de diseño: `/stats/clasificacion` devuelve los colores desde el servidor:

```
colors = {"Hidrometeorológico": "#3b82f6", "Geológico": "#f97316"}
```

Es discutible —mezcla presentación con datos— pero tiene la ventaja de que si apareciera una tercera clasificación, el respaldo `"#6b7280"` la cubre sin tocar el frontend. El dashboard actual ignora ese color y aplica su propia paleta validada, por lo que el punto es hoy discutible en la práctica además de en principio.

Los dos endpoints del dashboard interactivo

Endpoint	Qué devuelve
<code>GET /stats/eventos</code>	Las 3 958 filas, 14 campos cada una, en formato compacto
<code>GET /stats/dimensiones</code>	Catálogos de filtro derivados de los datos

`/stats/eventos` devuelve listas, no objetos. La respuesta trae los nombres de columna una sola vez y las filas como arreglos posicionales:

```
{
  "columnas": ["año", "mes", "estado", "clasificacion", "tipo", "duracion",
    "daños", "poblacion", "defunciones", "viviendas",
    "escuelas", "hospitales", "comercios", "cultivo"],
  "filas": [[2000, 12, "Ciudad de México", "Geológico",
    "Actividad Volcánica", 1, 115.8, 41000, 0, 0, 0, 0, 0, 0.0], ...],
  "total": 3958
}
```

Repetir catorce claves 3 958 veces añadiría más de 150 KB de nombres de campo redundantes. El frontend rehidrata las filas a objetos al recibirlas: cuesta unos milisegundos una vez y a cambio el resto del código se lee como `e.estado` en lugar de `fila[2]`.

La tabla se construye una sola vez, al arrancar. No por petición: el `DataFrame` es estático, así que recalcularlo en cada llamada sería trabajo repetido. Sigue el mismo patrón con que se cargan los artefactos del modelo.

Limpieza de columnas de texto sucio. Seis columnas del Excel llegan como `object` con marcadores de ausencia y basura de formato — `"SD"` , `"sd"` , `"19 362 "` , `" 13,637.00 \n"` —. La función `_a_numero()` quita todo lo que no sea dígito, punto o signo, y manda a `NaN` lo que quede vacío:

Columna	Filas convertibles a número	Total acumulado
Defunciones	3 942 / 3 958	4 697 personas
Viviendas dañadas	3 916 / 3 958	2 302 817 viviendas
Escuelas	3 900 / 3 958	43 955
Hospitales	3 899 / 3 958	3 231
Comercios	3 931 / 3 958	144 764
Área de cultivo (ha)	3 879 / 3 958	7 589 924

Los valores ausentes viajan como `null` de JSON, no como cero. La distinción importa: `json.dumps` de Python emite `NaN` para un flotante NaN, que **no es JSON válido** y hace fallar el `JSON.parse` del navegador. Las funciones `_entero_o_nulo()` y `_decimal_o_nulo()` existen precisamente para eso.

Un detalle de precisión que costó un error real. La primera versión redondeaba los daños a dos decimales. El daño no nulo más pequeño del conjunto es **0.00047 millones** (unos 466 pesos) y hay **76 eventos por debajo de 0.005**: redondear a dos decimales los mandaba a cero, y el histograma del dashboard reportaba 1 141 eventos "sin daño" donde en realidad hay 1 065. Se detectó comparando el histograma contra el conteo hecho con pandas. La salida usa cuatro decimales.

`/stats/dimensiones` devuelve el rango de años, los 32 estados, las 2 clasificaciones, los 9 tipos y el mapa tipo → clasificación, todo derivado de `stats_df` con `unique()`. Se deriva de los datos y no de una lista escrita a mano para que el frontend no pueda ofrecer un filtro inexistente ni omitir uno real.

Compresión de respuestas

```
app.add_middleware(GZipMiddleware, minimum_size=1024)
```

No es un adorno: es lo que hace viable mandar el detalle por evento al navegador.

	Bytes
<code>/stats/eventos</code> sin comprimir	354 232
<code>/stats/eventos</code> con gzip	40 086

El umbral de 1 KB evita gastar ciclos de CPU comprimiendo respuestas pequeñas como `/health`, donde la cabecera de gzip costaría más de lo que ahorra.

Cacheo de los endpoints de estadísticas

Los seis responden con `JSONResponse` y una cabecera explícita:

```
STATS_CACHE = "public, max-age=3600"
...
return JSONResponse(content=[...], headers={"Cache-Control": STATS_CACHE})
```

`public` autoriza a cachés intermedias (CDN, proxy) además del navegador. `max-age=3600` fija una hora de validez.

Es la elección correcta para estos datos y se justifica con precisión: los artefactos **solo cambian cuando alguien ejecuta el entrenamiento y despliega**. Entre despliegues, `/stats/kpis` devuelve exactamente el mismo JSON en cada petición. Sin cacheo, cada carga del dashboard dispara seis peticiones que recalculan seis `groupby` sobre 3 958 filas para producir un resultado idéntico. Con una hora de caché, el navegador sirve la segunda visita desde disco.

El commit `77dedb3` introdujo esta cabecera específicamente. Nótese que `/predict` no lleva `Cache-Control`, y es correcto: es un `POST` y cada combinación de entradas es distinta.

CORS: lista blanca en producción, cualquier puerto local en desarrollo

```
ORIGENES_PRODUCCION = [
    "https://desastres-naturales-gamma.vercel.app",
]

ORIGENES_DESARROLLO = r"http://(localhost|127\.0\.0\.1):\d+"

app.add_middleware(
    CORSMiddleware,
    allow_origins=ORIGENES_PRODUCCION,
    allow_origin_regex=ORIGENES_DESARROLLO,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

CORS (*Cross-Origin Resource Sharing*) es el mecanismo por el que un navegador decide si una página servida desde un origen puede hacer peticiones a otro. Sin él, la SPA en `localhost:5173` no podría llamar a la API en `127.0.0.1:8001`: puerto distinto es origen distinto.

Por qué lista blanca y no `["*"]` en producción. Con `allow_origins=["*"]` cualquier sitio web del mundo podría consumir la API desde el navegador de sus visitantes. Más concretamente: el comodín es **incompatible con `allow_credentials=True`** según la especificación de CORS —los navegadores rechazan esa combinación—, así que la lista explícita no es solo buena higiene, es un requisito técnico dada la configuración elegida.

Por qué una expresión regular en desarrollo. La versión anterior fijaba a mano `localhost:5173` y `127.0.0.1:5173`. El problema apareció en la práctica: Vite **no siempre levanta en 5173**. Si el puerto está ocupado —por otra instancia del mismo proyecto, por ejemplo— toma silenciosamente el siguiente libre, y entonces todas las peticiones del dashboard fallan con un error de CORS que no dice "puerto equivocado" sino algo mucho más opaco. Fijar un puerto obligaba a editar el backend cada vez que eso pasaba.

La expresión `http://(localhost|127\.0\.0\.1):\d+` acepta cualquier puerto, pero **solo de la máquina local**. Un atacante no puede servir una página desde `localhost` del visitante, así que no relaja nada en producción. Las dos configuraciones coexisten: `allow_origins` para los orígenes desplegados, `allow_origin_regex` para el desarrollo.

El costo operativo, que hay que conocer: cada nuevo origen de frontend exige editar `main.py` y **redesplegar el backend**. Un despliegue de vista previa de Vercel, que genera una URL nueva por cada rama, sería bloqueado. Nótese además que `127.0.0.1:5173` y `localhost:5173` están **ambos** listados: para el navegador son orígenes distintos aunque apunten a la misma máquina.

CAPÍTULO 7

El frontend

Pila tecnológica

Paquete	Versión	Papel
<code>react</code> / <code>react-dom</code>	<code>^19.2.0</code>	biblioteca de interfaz
<code>vite</code>	<code>^7.2.4</code>	servidor de desarrollo y empaquetador
<code>react-simple-maps</code>	<code>^3.0.0</code>	componentes de mapa sobre d3-geo
<code>d3-scale</code>	<code>^4.0.2</code>	escala de color continua del coroplético
<code>d3-geo</code>	<code>^2.0.2 (dev)</code>	usado solo por el script de verificación
<code>eslint</code>	<code>^9.39.1</code>	análisis estático

El archivo `frontend/.npmrc` contiene una sola línea:

```
legacy-peer-deps=true
```

Es necesaria porque `react-simple-maps@3` declara compatibilidad con React 16–18 en sus `peerDependencies`, y el proyecto usa React 19. Sin esa bandera, `npm install` aborta con un conflicto. La bandera indica a npm que resuelva las dependencias con el algoritmo permisivo de npm 6 en lugar de fallar. Funciona —el componente no usa API removidas en React 19— pero es deuda técnica declarada: el commit [7cc04d1](#) la introdujo explícitamente por este motivo.

Estructura y las dos pestañas

`App.jsx` es el componente raíz y mantiene una sola pieza de estado de navegación:

```
const [tab, setTab] = useState("predictor");
...
{tab === "dashboard" && <DashboardHistorico />}
{tab === "predictor" && ( ... )}
```

No hay enrutador. Es un condicional sobre una cadena. Para dos vistas sin URLs propias es una decisión proporcionada: añadir `react-router` sumaría una dependencia y complejidad de configuración para resolver un problema que no existe. **La contrapartida** —que el capítulo 11 registra— es que no se puede enlazar directamente al dashboard, ni el botón "atrás" del navegador cambia de pestaña.

Pestaña 1: el predictor

Selectores dependientes

El formulario tiene cinco campos, y dos de ellos están encadenados:

```
const tiposPorClasificacion = {
  Geológico: ["Actividad Volcánica", "Sismos", "Movimientos de Masa"],
  Hidrometeorológico: ["Sequía", "Ciclones", "Frío Extremo",
    "Lluvias e Inundaciones", "Viento Extremo", "Calor Extremo"],
};
```

Al cambiar la clasificación se limpia el tipo, para impedir un estado inconsistente:

```
if (name === "Clasificación_del_fenómeno") {
  setForm({ ...form, Clasificación_del_fenómeno: value, Tipo_de_fenómeno: "" });
  return;
}
```

Y mientras no haya clasificación elegida, el selector de tipo está **disabled**. Es una restricción del dominio codificada en la interfaz: no existe un "sismo hidrometeorológico".

Verificación importante: los 9 tipos de la interfaz coinciden **exactamente** —cadena por cadena, acento por acento— con los 9 tipos que el **OneHotEncoder** aprendió, y los 32 estados del selector coinciden con las 32 categorías del modelo (con **"Estado de México"** como única etiqueta que la API traduce). Si alguna difiriera por un acento, el one-hot la ignoraría en silencio y la predicción se degradaría sin aviso.

Presentación del resultado

Las tarjetas de resultado se rediseñaron. La versión anterior tenía una **incongruencia visible** entre lo que decía el texto y lo que mostraba la barra, y vale la pena documentarla porque el diagnóstico es más interesante que el arreglo.

El problema: la etiqueta y la barra no hablaban de lo mismo

El nivel cualitativo salía de unos cortes:

```
const getNivelImpacto = (valor) => {
  if (valor < 100) return { texto: "Bajo", color: "#22c55e" };
  if (valor < 500) return { texto: "Medio", color: "#f59e0b" };
  return { texto: "Alto", color: "#ef4444" };
};
```

y el ancho de la barra, de una fórmula **sin ninguna relación con ellos**:

```
Math.min((monto / 2000) * 100, 100)
```

Los dos mecanismos eran independientes, y el resultado era contradictorio:

Predicción	Etiqueta	Barra (fórmula anterior)	Lectura
99 MDP	Bajo	5.0 %	coherente
100 MDP	Medio	5.0 %	cruza de banda y la barra se mueve 0.05 %
500 MDP	Alto	25.0 %	dice "Alto" con la barra en un cuarto
2 000 MDP	Alto	100 %	saturada
84 207 MDP	Alto	100 %	idéntica a la de 2 000

Dos defectos de fondo. El primero: **el techo de 2 000 MDP es el percentil 98.76** de los eventos históricos, y el máximo real —84 207.02 MDP, el huracán Otis en Guerrero— es **42 veces mayor**. El 1.2 % de los eventos saturaba la barra sin distinguirse entre sí. El segundo: la escala era **lineal** sobre una distribución con siete órdenes de magnitud, el mismo problema que obligó al modelo a entrenar sobre `log1p`.

El arreglo: una escala por bandas con anclas fijas

La barra ya no se calcula con una fórmula global, sino recorriendo **bandas** cuyos bordes coinciden exactamente con los umbrales de la etiqueta:

Valor	Posición en el riel
0	0 %
100 MDP (borde Bajo → Medio)	40 %
500 MDP (borde Medio → Alto)	70 %
84 207.02 MDP (máximo histórico)	100 %

Dentro de cada banda se interpola en `log1p`, no linealmente:

```
const t = (Math.log1p(v) - Math.log1p(lo)) / (Math.log1p(hi) - Math.log1p(lo));
```

La invariante que esto garantiza: si el texto dice "Alto", la barra está pasada del 70 % *por construcción*, no por coincidencia. Los bordes de banda caen en 40 % y 70 % del riel, y ahí van dibujadas dos marcas verticales etiquetadas `$100 M` y `$500 M`, de modo que el lector ve dónde están los cortes en lugar de tener que deducirlos.

Predicción	Antes	Ahora
99 MDP	5.0 %	39.9 %
100 MDP	5.0 %	40.0 %
500 MDP	25.0 %	70.0 %
2 000 MDP	100 %	78.1 %
84 207 MDP	100 %	100 %

La invariante se comprobó por fuerza bruta sobre 400 002 valores repartidos entre las dos escalas, con muestreo denso en la cola baja: **cero incongruencias** entre la etiqueta y el llenado.

Por qué los umbrales NO se recalibraron por cuantiles

Es una decisión deliberada que conviene poder defender, porque la alternativa parece más rigurosa y no lo es.

Con los cortes actuales, el **89.4 %** de los eventos históricos cae en "Bajo". Eso parece un defecto de calibración, y la corrección aparentemente obvia sería mover los cortes a los terciles de la distribución. Se descartó: el percentil 33 del daño está por debajo de **0.2 MDP**, así que un corte por cuantiles obligaría a etiquetar como "Impacto Alto" un evento de 5 millones de pesos. **Es falso en pesos**. Los umbrales son *semánticos* —miden severidad económica absoluta—, no cuantiles; y que la mayoría de los desastres en México sean económicamente pequeños es un hecho del fenómeno, no un error de la escala. 1 065 de los 3 958 eventos registrados tienen daño exactamente cero.

El contexto distribucional no se pierde: se da **aparte**, en su propia línea.

La línea de percentil

Bajo la barra aparece el rango histórico del valor predicho:

Por encima del 95.5 % de los 3 958 eventos registrados (2000-2023).

Sale de una rejilla ECDF de veinte anclas incrustada en el componente, interpolada en **log1p**. Aquí sí se usan cuantiles, pero como **información separada**, sin contaminar la etiqueta de severidad. El lector obtiene las dos lecturas que necesita —"cuán caro es esto en pesos" y "cuán raro es esto"— sin que ninguna se disfrace de la otra.

La misma lógica se aplica a la tarjeta de población afectada, con cortes elegidos para caer en percentiles equivalentes, de forma que "Impacto Alto" signifique lo mismo en ambas tarjetas:

Escala	Corte Bajo → Medio	Percentil	Corte Medio → Alto	Percentil
Daño económico	100 MDP	89.46	500 MDP	95.53
Población afectada	10 000 personas	89.86	30 000 personas	94.97

La barra de progreso: dos estados, no un porcentaje fingido

Mientras la petición está en vuelo, **el mismo riel**, en el mismo sitio y con las marcas de banda ya dibujadas, muestra un barrido indeterminado; al llegar la respuesta se llena hasta la posición del valor con una transición de ancho y color. La animación respeta `prefers-reduced-motion`.

La decisión de diseño que hay que saber justificar es la de **no fingir un porcentaje de avance**. `/predict` es un solo viaje de red sin etapas intermedias medibles: no hay nada real que reportar. Una barra que avanzara al 90 % durante la carga y luego retrocediera al 12 % al aterrizar un impacto "Bajo" se leería como un fallo —y sería exactamente la misma incongruencia entre texto y llenado que este rediseño vino a eliminar. En su lugar, el encabezado muestra los segundos transcurridos, y pasados cuatro segundos añade que el servidor puede estar despertando de suspensión, que es la causa real de las esperas largas en alojamiento gratuito.

Formateo adaptativo

El componente formateaba a dos decimales fijos, lo que mostraba **\$0** para una predicción de 0.00047 MDP. Era, en pantalla, el mismo defecto de precisión que se había corregido en la API (ver el capítulo 7). Ahora los decimales se adaptan a la magnitud —cuatro por debajo de 0.01, tres por debajo de 1, dos por debajo de 1 000, ninguno por encima— y el cero se expresa con palabras, "*Sin daños económicos estimados*", acompañado de la nota de que 1 065 eventos históricos tampoco reportaron daño.

El formateo sigue usando la configuración regional mexicana:

```
Number(v).toLocaleString("es-MX", { ... })
```

Tarjetas derivadas de la respuesta

Las tarjetas ya no son una lista fija escrita a mano: se generan recorriendo las claves de `data.prediction`. Una métrica sin escala registrada se degrada a tarjeta sin barra en lugar de romper la pantalla. Es una precaución concreta: hay un rediseño en estudio que retiraría "**Población afectada**" de la respuesta de `/predict`, y con el código anterior eso habría dejado una tarjeta vacía.

La interfaz conserva el descargo de responsabilidad visible:

Estimación basada en modelos estadísticos. Puede variar según condiciones reales y calidad de los datos reportados.

Pestaña 2: el dashboard histórico

El dashboard se reconstruyó por completo. La versión anterior era **estática**: seis peticiones a seis endpoints de agregados fijos, sin un solo filtro, y el único gesto interactivo de todo el panel era el tooltip al pasar el ratón sobre el mapa. Mostraba seis de las treinta y una columnas del conjunto de datos y no mostraba el mes en ninguna parte, pese a que el mes es una de las siete variables de entrada del modelo.

La versión actual es un **tablero de filtrado cruzado**: cada gráfica es a la vez una salida (muestra datos) y una entrada (se hace clic sobre ella para filtrar).

La decisión de arquitectura: agregar en el navegador

Es la decisión de diseño que hay que saber defender, porque determina todo lo demás.

Un dashboard con filtros tiene dos arquitecturas posibles:

Enfoque	Cómo funciona	Costo por interacción
Agregación en el servidor	Cada filtro dispara peticiones nuevas; el backend agrupa y devuelve totales	Una ida y vuelta de red por cada clic (decenas o cientos de ms)
Agregación en el cliente	El navegador descarga el detalle una vez y recalcula en memoria	Ninguna; el recálculo es local

Se eligió la segunda, y la justificación es cuantitativa: el conjunto de datos tiene **3 958 filas**. Servido como lista de listas y comprimido con gzip, pesa **40 086 bytes**. Descargar cuarenta kilobytes una sola vez es menos tráfico que tres interacciones del usuario contra endpoints agregados.

Medición	Valor
Respuesta de <code>/stats/eventos</code> sin comprimir	354 232 bytes
Respuesta comprimida con gzip	40 086 bytes
Factor de compresión	8.8×
Filas transmitidas	3 958
Columnas por fila	14

Dos detalles hacen viable ese número. El primero es el **formato compacto**: las filas viajan como arreglos posicionales (`[2017, 9, "Oaxaca", ...]`) con la lista de nombres de columna aparte, en lugar de repetir las catorce claves 3 958 veces; eso reduce el JSON casi a la mitad. El segundo es `GZipMiddleware`, añadido al backend con umbral de 1 KB.

El límite de la decisión, que hay que reconocer. Este enfoque escala con el tamaño del conjunto de datos, no con el número de usuarios. Con 3 958 filas es claramente correcto. Con 400 000 filas —si la base se extendiera a nivel municipal, por ejemplo— el navegador tendría que descargar y mantener en memoria varios megabytes, y la agregación tendría que volver al servidor. Es una decisión correcta *para este tamaño de datos*, y el comentario en el código lo dice explícitamente para que quien lo herede no lo aplique a ciegas.

Los dos endpoints que consume

El dashboard hace exactamente **dos** peticiones al montarse, y ninguna más durante toda la sesión:

Endpoint	Qué devuelve
GET <code>/stats/dimensiones</code>	Catálogos para los controles: rango de años, los 32 estados, las 2 clasificaciones, los 9 tipos y el mapa tipo → clasificación
GET <code>/stats/eventos</code>	Las 3 958 filas con 14 campos cada una

Los catálogos se derivan de los datos, no de una lista escrita a mano en el frontend. Así el panel no puede ofrecer un filtro que no exista en el conjunto de datos, ni quedarse sin ofrecer uno que sí existe.

Los seis endpoints agregados anteriores (`/stats/kpis` , `/stats/evolucion-anual` , `/stats/top-estados` , `/stats/por-estado` , `/stats/clasificacion` , `/stats/top-eventos`) **siguen existiendo y funcionando**; el dashboard simplemente ya no los llama. Se conservaron por compatibilidad hacia atrás.

Organización del código

El componente anterior era un único archivo de 700 líneas que mezclaba constantes de maquetado, componentes de gráfica, lógica de mapa y composición. Se dividió en módulos con una responsabilidad cada uno:

Archivo	Responsabilidad
<code>dashboard/datos.js</code>	Carga de los dos endpoints y exportación a CSV
<code>dashboard/agregaciones.js</code>	Filtrado y agregación — funciones puras, sin React
<code>dashboard/tema.js</code>	Paleta, definición de métricas y formato de números
<code>dashboard/Controles.jsx</code>	Barra de filtros
<code>dashboard/Graficas.jsx</code>	Serie anual, rankings, estacionalidad, histograma, barra de proporción
<code>dashboard/Mapa.jsx</code>	Mapa coroplético
<code>dashboard/Tabla.jsx</code>	Tabla de detalle ordenable
<code>DashboardHistorico.jsx</code>	Composición y estado compartido del filtro

La separación no es cosmética. `agregaciones.js` no importa React ni toca el DOM: recibe datos y devuelve datos. Eso permite memorizar sus resultados con `useMemo` y —más importante— **probarlo sin montar un navegador**, que es exactamente lo que hace la suite de pruebas descrita en el capítulo 11.

La regla del filtrado cruzado

Es el detalle de diseño más fácil de equivocar, y el que hay que saber explicar.

La función de filtrado acepta un tercer argumento:

```
export function filtrar(eventos, filtro, omitir = null) { ... }
```

omitir excluye **una** dimensión del filtrado, y cada gráfica pasa la suya:

Gráfica	Omite
Mapa y ranking de estados	"estados"
Ranking de fenómenos	"tipos"
Patrón estacional	"meses"
Reparto por clasificación	"clasificaciones"

Por qué. Sin esta regla, al seleccionar "Ciclones" el ranking de fenómenos se quedaría con **una sola barra** y el mapa con **un solo estado pintado**: dos gráficas que ya no comparan nada, porque se han filtrado a sí mismas hasta quedarse sin puntos de comparación. Con la regla, el ranking sigue mostrando los nueve fenómenos —recortados por año, estado y mes— y marca el seleccionado con una palomita; el mapa sigue pintando el país entero y marca la selección con un contorno oscuro en lugar de cambiar el relleno, porque el relleno ya está ocupado codificando la magnitud.

La excepción. El rango de años **nunca** se omite. No es una categoría que se compare contra sus hermanas, sino un acotamiento del periodo que se está mirando. Si la serie anual omitiera su propio filtro, arrastrar sobre ella para acotar los años no tendría ningún efecto visible.

Los controles

Control	Comportamiento
Métrica	Cinco opciones: daños, población afectada, defunciones, viviendas dañadas y número de eventos. Cambia qué miden todas las gráficas a la vez , mapa incluido
Rango de años	Deslizador de dos pulgares (dos <code>input range</code> superpuestos, con <code>pointer-events</code> devuelto solo a los pulgares)
Estado, fenómeno, mes	Selector múltiple propio con búsqueda. Un <code><select multiple></code> nativo obliga a mantener Ctrl presionado y no permite buscar; con 32 estados eso no funciona
Clasificación	Dos chips de dos estados
Píldoras	Cada filtro activo aparece como una píldora con × para quitarlo individualmente
Limpiar filtros	Vuelve al estado neutro; se deshabilita cuando ya lo está

Además, **cada gráfica filtra al hacer clic**: un estado del mapa, una barra de fenómeno, un mes del patrón estacional, un segmento de la barra de clasificación. Y la serie anual acepta **arrastre** para acotar el rango de años directamente sobre la gráfica.

Los controles viven todos en una sola fila superior, no repartidos entre las tarjetas, porque el usuario tiene que poder ver de un vistazo qué recorte está mirando.

Las gráficas

Se mantuvo la decisión original de dibujar el SVG a mano, sin biblioteca de gráficas: cero dependencias añadidas, control total de la interacción y ningún conflicto de compatibilidad con React 19. Pero se corrigieron las carencias que el diseño anterior tenía:

Carencia anterior	Estado actual
Sin tooltips en ninguna gráfica	Todas tienen capa de hover; la serie anual además lleva línea de cursor
<code>Math.max(...[])</code> devolvía <code>-Infinity</code> con datos vacíos	Todas manejan el conjunto vacío con un mensaje explícito
Sin forma de ver el valor exacto de un punto	Tooltip con valor, número de eventos y participación porcentual

Y se añadieron dos gráficas que antes no existían:

Patrón estacional. Doce barras, una por mes. Es la dimensión que más dice del fenómeno y la que el panel anterior ignoraba por completo. También es la que **conecta el dashboard con el modelo**:

`mes_sin` y `mes_cos` son dos de las siete variables de entrada, y aquí se ve por qué importan. La concentración es fuerte: septiembre reúne 684 eventos y diciembre 57.

Distribución de la magnitud del daño. Un histograma con bandas en potencias de diez. Es la gráfica que **explica el modelo**:

Banda (millones de pesos)	Eventos
0 (sin daño registrado)	1 065
0 – 0.1	736
0.1 – 1	735
1 – 10	592
10 – 100	413
100 – 1 000	318
1 000 – 10 000	87
> 10 000	12

La montaña apilada a la izquierda y la cola de doce eventos a la derecha son, literalmente, la razón de ser de la transformación `log1p` del capítulo 6. El cero vive en su propia banda porque `log(0)` no existe y porque "no hubo daño económico registrado" es una categoría distinta de "hubo poco".

Cambios de forma por criterio de visualización

Dos gráficas se reemplazaron por razones de lectura, no de gusto:

La dona de clasificación → una barra de proporción. La clasificación tiene exactamente dos categorías, y el reparto es de 92 % / 8 % en eventos. Una dona de dos rebanadas obliga al lector a comparar ángulos para leer un dato que una barra dice directamente. Además, una dona de dos segmentos gasta un espacio que rinde más como gráfica de barras.

Se añadió el ranking de fenómenos, que antes no existía. Son nueve categorías, y aquí aparece una regla que conviene explicitar: **nueve categorías superan las ocho ranuras de cualquier paleta categórica segura para daltonismo**. La solución no es generar un noveno color —dos tonos generados serían indistinguibles bajo deuteranopía—, sino reconocer que es **una sola serie**: la longitud de la barra codifica la magnitud y el color no codifica nada. Un solo color para las nueve barras.

Por la misma razón se eliminó el resaltado del primer lugar del ranking de estados con un azul más oscuro: coloreaba según el rango, lo que duplica en el canal de color una información que la longitud de la barra ya da. Ese tono oscuro se reasignó a marcar la **selección**, que sí es información nueva.

La paleta se validó con un verificador automático de separación bajo daltonismo. El par de tonos que aparece simultáneamente en pantalla (azul `#2a78d6` y naranja `#eb6834`) obtiene una separación ΔE de 24.7 bajo protanopía y 33.6 en visión normal, muy por encima del umbral de 8.

La tabla de detalle

La tabla anterior mostraba los diez eventos más caros y nada más. La actual muestra **el recorte filtrado completo**, ordenable por cualquiera de sus nueve columnas, paginada y exportable a CSV con codificación BOM para que Excel abra los acentos correctamente.

Cumple además una función de accesibilidad: es la **vista de tabla** que respalda a las gráficas. Quien no distinga los tonos del mapa puede leer ahí exactamente los mismos números. Los valores ausentes se muestran como `s/d` y se ordenan siempre al final, porque la ausencia de dato no es un cero.

Tres estados explícitos, ningún dato simulado

La versión anterior inicializaba el estado de React con seis constantes `MOCK_*` de cifras inventadas. Si la API no respondía, el panel mostraba **1 842 eventos y 524 300.5 MDP** —números falsos y plausibles— junto a un mensaje de error pequeño. Esas constantes se eliminaron por completo.

El componente ahora renderiza tres estados mutuamente excluyentes:

Estado	Qué muestra
Cargando	Mensaje de carga, ninguna cifra
Error	Mensaje explícito nombrando el valor de <code>VITE_API_URL</code> , ninguna cifra
Con datos	Los datos reales

En ningún momento puede aparecer un número que no venga de la API.

Los indicadores

Seis tarjetas en lugar de cuatro, añadiendo defunciones y viviendas dañadas, que estaban en los datos y no se mostraban. Cuando hay algún filtro activo, cada tarjeta añade **qué porcentaje del total nacional** representa el recorte: sin esa referencia, un número filtrado se puede leer como si fuera el total.

Junto a los indicadores, cuando la métrica es daño económico, aparece una nota fija con el contraste entre mediana y promedio. No es decoración: es el hallazgo central del conjunto de datos, y sin él un lector concluiría que el desastre promedio en México cuesta 168.8 millones de pesos, cuando el desastre **típico** cuesta 0.16 millones.

El mapa coroplético

Composición

```
<ComposableMap projection="geoMercator"
  projectionConfig={{ scale: 1600, center: [-102, 24] }}>
  <Geographies geography="/mexico.geojson">
    ({ geographies }) => geographies.map((geo) => { ... })
  </Geographies>
</ComposableMap>
```

La proyección es Mercator centrada en $(-102^\circ, 24^\circ)$ —aproximadamente el centro geográfico de México— con escala 1600, ajustada empíricamente para que el país llene el contenedor.

La escala de color

La versión anterior usaba una interpolación **lineal** entre rojo claro y rojo oscuro:

```
const colorScale = scaleLinear().domain([0, maxVal]).range(["#fecaca", "#f1d1d1"]);
```

Tenía un defecto de lectura serio. El máximo lo fija Guerrero con 118 142 MDP acumulados; Sinaloa, décimo de la lista con 14 458 MDP, caía en el 12 % de la escala, y los estados de la mitad inferior quedaban indistinguibles del rosa más pálido. **La misma cola pesada que obligó a usar `log1p` en el modelo perjudicaba la lectura del mapa.**

La versión actual corrige las dos cosas: la no linealidad y el tono.

```
const escala = scaleLinear()
  .domain([0, Math.sqrt(maxValor)])
  .range([0, RAMPA_SECUENCIAL.length - 1])
  .clamp(true);
```

La escala es en raíz cuadrada, no lineal. Comprime la cola sin aplastar el extremo bajo como haría un logaritmo puro, y a diferencia del logaritmo admite el cero sin caso especial. Los estados de la mitad inferior recuperan variación visible.

La rampa es discreta y de un solo tono (ocho pasos de azul, de claro a oscuro). Un solo tono es obligatorio en codificación de magnitud: en un arcoíris, saltos de color no corresponden a saltos de valor, y el lector no puede ordenar los tonos mentalmente. Se pasó de rojo a azul por coherencia con el resto del panel y para que el color del mapa no se confunda con una señal de alerta.

Los estados sin datos se pintan en gris (#f3f4f6) y no en el extremo claro de la escala, para distinguir "cero daño" de "sin información". La leyenda muestra los ocho escalones discretos, no un degradado continuo: una escala no lineal **exige** explicitar la correspondencia entre color y valor.

Interacción del mapa

El mapa dejó de ser una figura y pasó a ser un control:

Gesto	Efecto
Clic sobre un estado	Lo añade o lo quita del filtro; todo el tablero se recalcula
Rueda del ratón	Acerca y aleja (<code>ZoomableGroup</code> , de 1× a 8×)
Arrastre	Desplaza el mapa
Botón "Centrar mapa"	Vuelve a la vista inicial; solo aparece si el usuario movió el mapa

La selección se marca con un **contorno** oscuro y grueso, no cambiando el relleno, porque el relleno ya está ocupado codificando la magnitud. Y el mapa **ignora su propio filtro de estados** por la regla de filtrado cruzado explicada arriba: si respetara la selección, elegir Veracruz apagaría los otros treinta y uno y el mapa dejaría de servir para comparar.

El join por nombre de estado

Este es el punto donde el frontend y los datos se encuentran, y donde una divergencia de un solo carácter rompería el mapa en silencio:

```
const GEO_NAME_MAP = {
  "Coahuila de Zaragoza": "Coahuila",
  "Distrito Federal": "Ciudad de México",
  "Michoacán de Ocampo": "Michoacán",
  "Veracruz de Ignacio de la Llave": "Veracruz",
};
...
const nombre = GEO_NAME_MAP[geo.properties.name] ?? geo.properties.name;
const info = lookup[nombre] ?? { daños: 0, poblacion: 0, total_eventos: 0 };
```

El GeoJSON usa nombres oficiales largos, y además `"Distrito Federal"` —una denominación anterior a 2016—. La tabla los traduce a los nombres normalizados que produjo `clean_base()`.

Verificación realizada para este manual. Se cargó `mexico.geojson`, se aplicó `GEO_NAME_MAP` a los 32 nombres y se comparó contra los 32 nombres presentes en `data.joblib`:

Comprobación	Resultado
Polígonos en el GeoJSON	32
Estados en los datos	32
En los datos pero sin polígono	ninguno
Con polígono pero sin datos	ninguno

El join es completo hoy. Pero es un acoplamiento por cadena de texto entre tres artefactos independientes —el Excel, el GeoJSON y el diccionario en el componente— y sigue sin una prueba que lo verifique: la suite de pruebas añadida (ver L6) cubre las agregaciones, no el join de nombres contra el GeoJSON. Nótese además el contraste: la tabla del frontend traduce "Distrito Federal" mientras que la tabla del backend (`CENSUS_T0_BASE`) no lo hace, porque INEGI ya usa "Ciudad de México". Son dos diccionarios de traducción distintos, mantenidos en lugares distintos, que deben permanecer mutuamente coherentes.

La simplificación del GeoJSON

El commit [5a86102](#) lleva por título "*Simplificar mexico.geojson: 9.5 MB → 360 KB (26×)*". El archivo actual pesa **368 501 bytes** verificados.

El GeoJSON original tenía las fronteras estatales con precisión cartográfica completa: 252 558 líneas de coordenadas. Servir 9.5 MB para pintar un mapa de 1 100 píxeles de ancho es desperdicio puro —a esa resolución, la diferencia entre un litoral con diez mil vértices y uno con cuatrocientos es invisible—, y tenía tres costos concretos: transferencia de red, tiempo de parseo del JSON en el hilo principal del navegador, y tiempo de cálculo de la proyección para cada vértice.

La simplificación (por ejemplo con `mapshaper`, que aplica el algoritmo de Visvalingam o Douglas-Peucker) elimina vértices preservando la forma reconocible.

El script de verificación de orientación

La simplificación trae un riesgo sutil, y el proyecto lo trató con seriedad notable: existe `frontend/scripts/verifica-geojson.mjs`, invocable con `npm run verify:geojson`.

El problema que detecta está documentado en el propio archivo. Las herramientas de simplificación suelen escribir los anillos de polígono en la convención de *Shapefile* (anillo exterior en sentido horario), mientras que `d3-geo` exige la **contraria** (RFC 7946, antihorario). Con la orientación invertida, d3 interpreta cada estado como su complemento —"todo el globo menos el estado"— y **el mapa se pinta como una mancha uniforme, sin lanzar ningún error.**

La detección es elegante:

```
const MEDIA_ESFERA = 2 * Math.PI; // ningún estado real se acerca a esto
const invertidos = geojson.features.filter((f) => geoArea(f) > MEDIA_ESFERA);
```

`d3.geoArea()` devuelve el área en estereorradianes. Un polígono invertido devuelve $\sim 4\pi$ (12.566, la esfera completa) en lugar del área real del estado. El script sale con código 1 si encuentra alguno.

Detecta precisamente un fallo silencioso —la clase de error más difícil de encontrar por inspección—, y durante el desarrollo cumplió su función: el commit `3e32f4f` ("*corregir orientación de anillos del geojson de estados*") existe porque el script encontró el problema.

El episodio vale la pena contarlo completo, porque es el mejor argumento del manual a favor de las pruebas automatizadas. La simplificación del GeoJSON entró el **4 de julio de 2026** (commit `5a86102`) e invirtió el winding de los 32 polígonos. La corrección entró el **6 de septiembre de 2026** (commit `3e32f4f`). Entre una y otra pasaron **64 días** durante los cuales el mapa de `main` estuvo roto: `d3-geo` interpretaba cada estado como su complemento y pintaba el país como una mancha uniforme.

Los dos rasgos que hicieron posible ese lapso son exactamente los que describen la clase de error más cara de un proyecto:

1. **No lanzaba ningún error.** Ni excepción, ni advertencia en consola, ni petición fallida. Todo el código se ejecutaba correctamente; solo el resultado visual era absurdo.
2. **Nada automatizado lo comprobaba.** Detectarlo exigía que una persona abriera la pestaña del dashboard y mirara el mapa, y durante dos meses nadie con contexto para reconocer el fallo lo hizo.

La corrección no fue solo arreglar el archivo, sino escribir el guardarraíl que impide que vuelva a ocurrir sin avisar. Ese es el patrón correcto: cuando un fallo silencioso se manifiesta, el arreglo incluye el detector.

Junto con `npm run prueba:agregaciones` (ver L6), son los dos únicos mecanismos de verificación automatizada del repositorio. Ambos viven en el frontend; el backend sigue sin ninguno.

Configuración de entorno

```
VITE_API_URL=http://127.0.0.1:8001
```

Vite expone al código del navegador únicamente las variables con prefijo `VITE_`, como `import.meta.env.VITE_API_URL`. Es una barrera de seguridad: impide que un secreto del entorno acabe empaquetado en el bundle por descuido.

Advertencia importante: el valor se **incrusta en el bundle en tiempo de compilación**, no se lee en ejecución. Cambiar la URL de la API en producción exige **recompilar y redesplegar** el frontend, no basta con cambiar una variable de entorno del servidor.

CAPÍTULO 8

Cómo se ejecuta

Nota. El archivo `CLAUDE.md` del repositorio documenta los comandos en sintaxis de PowerShell (Windows), reflejo del entorno donde nació el proyecto. Este capítulo da los equivalentes para **Linux (EndeavourOS)**, que es el entorno de desarrollo actual.

Requisitos previos

Requisito	Versión	Comprobación
Python	3.11 (fijado en <code>runtime.txt</code> : 3.11.9)	<code>python3 --version</code>
Node.js	18 o superior	<code>node --version</code>
npm	9 o superior	<code>npm --version</code>

Puesta en marcha del backend

```
cd ~/Documentos/proyectos/desastres-naturales/backend

# Crear el entorno virtual (solo la primera vez)
python3.11 -m venv .venv

# Activarlo
source .venv/bin/activate

# Instalar dependencias con versiones exactas
pip install -r requirements.txt

# Arrancar la API en el puerto que el frontend espera en local
uvicorn app.main:app --reload --port 8001
```

Comprobaciones inmediatas:

```
curl http://127.0.0.1:8001/health
curl -X POST http://127.0.0.1:8001/predict \
  -H "Content-Type: application/json" \
  -d
'{"Año":2023,"Mes":10,"Clasificación_del_fenómeno":"Hidrometeorológico","Tipo_de_fenómeno":"Ciclones","Estado":"Guerrero"}'
```

La documentación interactiva generada por FastAPI está en <http://127.0.0.1:8001/docs>.

Sobre `--reload`: reinicia el servidor ante cualquier cambio en el código. Es para desarrollo; en producción se omite y se usan varios *workers*.

Sobre el puerto 8001: no es arbitrario. `frontend/.env` apunta a `http://127.0.0.1:8001`. Si se arranca Uvicorn en el 8000 por omisión, el frontend no encontrará la API.

Reentrenamiento de los modelos

```
cd ~/Documentos/proyectos/desastres-naturales/backend
source .venv/bin/activate
python train_model_simple.py
```

Salida esperada:

```
Filas originales: 3967
Filas para estadísticas (dashboard): 3958

Métricas temporales (train <2020, test >=2020):
  Modelo A (daños): {"R2_log": 0.1661, "MAE": 209.09, "MedAE": 1.6924, "n_train": 3226,
"n_test": 732}
  Modelo B (población): {"R2_log": 0.1033, "MAE": 4380.21, "MedAE": 49.6169, "n_train": 3191,
"n_test": 732}

Artifacts guardados en: ../backend/app/artifacts
- model_danos.joblib
- model_poblacion.joblib
- preprocessor.joblib
- poblacion_estatal.joblib
- data.joblib
- metadata.json
```

Es necesario ejecutarlo cuando se edita `Base.xlsx` o `Poblacion_01.xlsx`, cuando se cambia la lista de features o los hiperparámetros, o cuando se actualiza `scikit-learn` / `numpy` (los artefactos van atados a la versión que los creó). **No** es necesario para cambios en la API o el frontend.

Tarda unos segundos y **sobrescribe los seis artefactos sin conservar copia**.

Puesta en marcha del frontend

```
cd ~/Documentos/proyectos/desastres-naturales/frontend

npm install          # .npmrc fuerza legacy-peer-deps
npm run dev          # servidor de desarrollo en http://localhost:5173
```

Otros comandos disponibles:

```

npm run build           # compilación de producción a dist/
npm run preview         # sirve dist/ localmente para revisarla
npm run lint            # ESLint
npm run verify:geojson  # verifica la orientación de los anillos del GeoJSON

```

Orden de arranque

El backend primero. Si el frontend arranca antes, el dashboard mostrará el mensaje de error (y los datos simulados). Basta recargar la página una vez que la API responda.

Despliegue

Frontend en Vercel

El frontend está desplegado en <https://desastres-naturales-gamma.vercel.app>.

Ajuste	Valor
Framework	Vite
Directorio raíz	frontend
Comando de compilación	npm run build
Directorio de salida	dist
Variable de entorno	VITE_API_URL = URL pública del backend

Como `VITE_API_URL` se incrusta al compilar, **cambiarla exige un redespiegue completo**, no solo actualizar la variable en el panel de Vercel.

Backend

El backend se aloja por separado (Vercel no es adecuado para un servicio que mantiene 750 KB de modelos en memoria y responde con latencia de milisegundos). El commit `066a0ea` menciona Render, y `runtime.txt` con `python-3.11.9` es justamente el mecanismo por el que Render y Heroku fijan la versión de Python.

Comando de arranque típico:

```
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Requisitos de despliegue:

1. **Publicar el directorio `backend/app/artifacts/` completo.** Sin él la API arranca pero responde con la clave `"error"` en cada predicción. Está fuera de `.gitignore`, es decir, versionado en el repositorio.
2. **Añadir el dominio del frontend a `allow_origins` en `main.py` y redespiegue.**

3. Respetar las versiones fijadas de `numpy` y `scikit-learn`.

Lista de verificación posterior al despliegue

```
curl https://<backend>/health           # los cinco *_loaded deben ser true
curl https://<backend>/stats/kpis       # total_eventos debe ser 3958
```

Y en el navegador, sobre el frontend desplegado: abrir la consola y comprobar que no hay errores de CORS, y que los indicadores muestran 3 958 eventos y no 1 842 (que sería la señal de que se están viendo los datos simulados).

CAPÍTULO 9

Alcance del sistema

Este capítulo delimita las fronteras. Enunciarlas con precisión es parte del trabajo: un sistema cuyo alcance no está definido no puede evaluarse.

Lo que el sistema SÍ hace

1. **Estima el impacto esperado condicionado al tipo de evento.** Dada una combinación de clasificación, tipo de fenómeno, estado, mes y año, devuelve una estimación de daño económico en millones de pesos y de población afectada en personas.
2. **Opera estrictamente ex-ante.** Ninguna de sus entradas requiere que el evento haya ocurrido. Todas son conocibles con antelación.
3. **Captura la estacionalidad.** Aprendió, sin que se le indicara, que los ciclones en Guerrero causan siete veces más daño en octubre que en marzo.
4. **Diferencia por estado y por fenómeno.** Distingue el perfil de riesgo de Guerrero del de Aguascalientes, y el de un ciclón del de una helada.
5. **Presenta el histórico consolidado.** El dashboard integra 3 958 eventos de 24 años en indicadores, series temporales, rankings y un mapa por estado —una consolidación que hoy no existe en una interfaz pública.
6. **Es reproducible de extremo a extremo.** Con `random_state=42` y versiones fijadas, cualquiera puede regenerar los artefactos y obtener exactamente las mismas métricas publicadas en `metadata.json`.
7. **Se autodocumenta.** `/health` expone el modo, las features y las métricas; `/docs` genera la especificación OpenAPI completa.

Lo que el sistema NO hace

Con la misma precisión, y esto es lo que hay que poder decir sin titubear:

1. **No pronostica ocurrencia.** No dice si habrá un ciclón, ni cuándo, ni dónde. Su respuesta es siempre condicional: *dado que* ocurre un evento de este tipo. Determinar si ocurrirá es meteorología y sismología, disciplinas con instrumentos —modelos numéricos de predicción del tiempo, redes sismográficas— completamente ajenos a este trabajo.
2. **No es un sistema de alerta temprana.** No monitorea nada en tiempo real, no se conecta a ninguna fuente de datos en vivo, no emite avisos. Los sistemas de alerta en México son el SASMEX para sismos y los avisos del Servicio Meteorológico Nacional. Este sistema es una herramienta de planeación estadística, no de operación en emergencia.

3. **No modela la intensidad del evento.** Un huracán categoría 1 y uno categoría 5 en el mismo estado y mes reciben **la misma predicción**, porque el modelo no tiene forma de distinguirlos. Es la limitación más importante y se desarrolla en el capítulo 11.
4. **No baja de la escala estatal.** La unidad geográfica mínima es el estado. No dice qué municipios, ni qué colonias, ni qué infraestructura específica. La columna **Municipios Afectados** existe en el Excel pero está en texto libre con 2 436 valores distintos y no se usa.
5. **No cuantifica su propia incertidumbre.** Devuelve un número, no un intervalo. No hay barra de error, ni percentiles, ni probabilidad. El sistema no sabe distinguir entre una predicción bien fundamentada (ciclones, 241 observaciones) y una casi sin sustento (actividad volcánica, 3 observaciones), y **presenta ambas con la misma apariencia de autoridad.**
6. **No debe usarse para agregar totales.** Como se midió en el capítulo 6, la suma de las predicciones sobre el conjunto de prueba equivale al **8.9 %** del daño real. Las predicciones sirven para **comparar escenarios entre sí**, no para construir un presupuesto agregado.
7. **No estima pérdida de vidas.** La columna **Defunciones** existe en el Excel y no se modela. Fue una decisión de alcance; añadir un tercer modelo sería técnicamente directo, pero la responsabilidad de publicar una estimación de muertes es de otra naturaleza.
8. **No ajusta por inflación.** Un daño de 100 millones de pesos de 2001 y uno de 100 millones de 2023 se tratan como iguales, aunque su poder adquisitivo difiera en más del doble.
9. **No se reentrena solo.** No hay tarea programada, ni disparador, ni supervisión de deriva. El modelo se queda como está hasta que una persona ejecuta el script.
10. **No tiene control de acceso.** La API es completamente abierta: sin autenticación, sin claves, sin límite de peticiones. Aceptable para un proyecto académico; inadmisible en producción.

La formulación precisa de lo que responde

Poniéndolo en una sola frase, que es la que conviene tener preparada para la defensa:

El sistema estima el **impacto esperado**, en escala logarítmica, de la **clase de eventos** históricamente registrados que comparten un tipo de fenómeno, un estado y un mes; **no** predice la ocurrencia de ningún evento concreto, **no** conoce su intensidad, y sus estimaciones tienen valor **comparativo** entre escenarios, no valor de pronóstico puntual.

CAPÍTULO 10

Limitaciones y áreas de oportunidad

Este es el capítulo más útil para defender el proyecto. Cada limitación se enuncia con su evidencia, se explica su consecuencia y se propone una mejora concreta y viable.

L1. Tamaño y desbalance del conjunto de datos

Evidencia. 3 958 filas útiles. Distribución por tipo de fenómeno:

Tipo	n	%
Lluvias e Inundaciones	2 441	61.5 %
Frío Extremo	412	10.4 %
Calor Extremo	292	7.4 %
Movimientos de Masa	241	6.1 %
Ciclones	241	6.1 %
Sequía	155	3.9 %
Viento Extremo	124	3.1 %
Sismos	58	1.5 %
Actividad Volcánica	3	0.1 %

Y por clasificación: 92.4 % hidrometeorológico contra 7.6 % geológico.

Consecuencia. Con `min_samples_leaf=20`, una hoja del árbol necesita al menos 20 observaciones. Los 3 registros de actividad volcánica **no pueden formar una hoja propia**: quedan absorbidos en un nodo con otros fenómenos. La predicción del sistema para actividad volcánica es, en la práctica, el promedio de un grupo heterogéneo. Lo mismo, en menor grado, para sismos. Esto se manifiesta en resultados contraintuitivos verificables: el modelo estima **89.99 MDP para un sismo en Aguascalientes** y **47.35 MDP para un sismo en la Ciudad de México**, invirtiendo el riesgo sísmico real de ambas entidades.

Mejora propuesta. Tres acciones, en orden de costo:

- *Inmediata (cero costo de datos):* excluir del formulario los tipos con menos de 30 observaciones, o marcarlos en la interfaz con una advertencia explícita ("estimación basada en 3 registros históricos").
- *Media:* extender la serie hacia atrás. CENAPRED publica el impacto socioeconómico desde 1980; incorporar 1980–1999 casi duplicaría la muestra y aumentaría sustancialmente el número de sismos, incluido 1985.
- *Avanzada:* incorporar el catálogo del Servicio Sismológico Nacional para eventos sísmicos y el de la base internacional EM-DAT como referencia cruzada.

L2. Granularidad estatal, no municipal

Evidencia. La unidad de análisis es el estado. La columna `Municipios Afectados` existe pero tiene 2 436 valores distintos en texto libre: `"Varios Municipios"`, `"81 municipios"`, `"SD"`, listas separadas por comas con saltos de línea incrustados, y nombres con y sin acentos.

Consecuencia. Se pierde toda la heterogeneidad interna. Un ciclón que devastó la costa de Guerrero y no tocó la Montaña se registra como "Guerrero", y el modelo aprende un promedio que no describe bien ninguna de las dos regiones. Como los estados grandes son internamente muy diversos, esto añade varianza irreducible al objetivo.

Mejora propuesta. Normalizar `Municipios Afectados` contra el catálogo de claves geoestadísticas de INEGI (INEGI publica el catálogo completo de claves municipales). Con un emparejamiento aproximado por similitud de cadenas y revisión manual de los casos ambiguos, se podría expandir cada evento a una fila por municipio afectado y unir el conteo de población municipal. Multiplicaría el tamaño efectivo de la muestra y permitiría al modelo aprender exposición local. Es la mejora de mayor rendimiento por unidad de esfuerzo entre todas las de esta lista, y no requiere ninguna fuente externa nueva más allá del catálogo.

L3. Ausencia de intervalos de confianza

Evidencia. `/predict` devuelve un número escalar por objetivo. No hay `"intervalo"`, ni `"percentil_10"`, ni `"desviación"`.

Consecuencia. Doble problema. Primero, el usuario no puede distinguir una estimación con respaldo de una casi inventada: "Ciclones en Guerrero" (241 observaciones, patrón fuerte) y "Actividad Volcánica en Tlaxcala" (3 observaciones en todo el país) se presentan idénticamente, con la misma tipografía y la misma barra de progreso. Segundo, para el uso previsto —dimensionar reservas financieras— lo que hace falta es precisamente un percentil alto ("¿cuál es el escenario del 95 %?"), no una estimación central.

Mejora propuesta. Es directa con las herramientas ya presentes:

```
# Tres modelos de cuantil en lugar de uno de media
modelos = {q: HistGradientBoostingRegressor(loss="quantile", quantile=q,
                                             random_state=42).fit(X, np.log1p(y))
            for q in (0.1, 0.5, 0.9)}
```

`HistGradientBoostingRegressor` soporta `loss="quantile"` de forma nativa. Con tres modelos se obtiene un intervalo del 80 % sin cambiar nada de la arquitectura. La respuesta pasaría a:

```
{"prediction": {"Total de daños (millones de pesos)":
               {"p10": 12.4, "p50": 639.9, "p90": 8420.1}}}
```

Complementariamente, exponer `n_observaciones` de la combinación consultada permitiría a la interfaz mostrar un indicador de confianza.

L4. Ausencia de variables físicas del evento

Evidencia. Las features son cinco: clasificación, tipo, estado, año y mes. Ninguna describe el evento físicamente.

Consecuencia. Es la limitación de mayor impacto en el desempeño, y la propia especificación de diseño la identifica como "el mayor techo de mejora". El modelo no puede distinguir el huracán Otis (categoría 5, 84 207 MDP) de una tormenta tropical menor en el mismo estado y mes. Como consecuencia directa, no puede anticipar los eventos extremos, que son exactamente los que concentran el 66.7 % del daño total. El percentil 95 del error absoluto (440.62 MDP) y el percentil 99 (1 820.10 MDP) son la medida de este techo.

Mejora propuesta, con fuentes concretas y disponibles públicamente:

Fenómeno	Variable a incorporar	Fuente
Ciclones	categoría Saffir-Simpson, presión central, distancia del ojo a la costa	IBTrACS (NOAA), archivo del SMN
Sismos	magnitud, profundidad, distancia epicentral	Servicio Sismológico Nacional (UNAM), USGS
Lluvias e Inundaciones	precipitación acumulada en 24/72 h	CHIRPS, ERA5, red de estaciones CONAGUA
Sequía	índice del Monitor de Sequía de México	CONAGUA, SMN
Frío / Calor Extremo	anomalía de temperatura respecto a la normal climatológica	ERA5, normales climatológicas del SMN

El obstáculo no es técnico sino de integración: cada fuente exige un emparejamiento espacio-temporal con los eventos de `Base.xlsx`, y los eventos no tienen coordenadas. Una implementación por etapas —empezando solo por ciclones, que son 241 eventos y concentran 7 de los 10 más costosos— sería un proyecto acotado con alta probabilidad de mejorar el R^2 de forma sustancial.

L5. Daños en pesos nominales, sin deflactar

Evidencia. El objetivo son pesos corrientes del año del evento, sin ajuste. La serie abarca 2000–2023, un periodo en el que el INPC de México acumuló una inflación superior al 130 %.

Consecuencia. 100 millones de pesos de 2001 y 100 millones de 2023 son la misma cifra para el modelo, aunque el primero represente más del doble de capacidad de compra. Esto introduce una **tendencia espuria**: los daños recientes parecen mayores solo por el efecto del nivel de precios. Y como `Año` es una feature, el modelo probablemente esté usando el año en parte como sustituto de la inflación, lo que además ensucia su capacidad de capturar tendencias reales de exposición.

Mejora propuesta. Deflactar a pesos constantes de un año base:

```
inpc = pd.read_csv("inpc_anual.csv") # serie del INEGI/Banxico
df = df.merge(inpc, on="Año")
df["danos_reales_2023"] = df[TARGET_DANOS] * (inpc_2023 / df["inpc"])
```

Es una mejora de bajo costo —el INPC anual son 24 números públicos— y alto valor interpretativo: el dashboard pasaría a mostrar series comparables entre décadas. Cabe advertir que probablemente **no** mejore mucho el R^2 , porque **Año** ya absorbe parte del efecto; el beneficio principal es de corrección conceptual y de honestidad en las series históricas.

L6. Cobertura de pruebas parcial — *parcialmente resuelta*

Situación original. El repositorio no tenía ni una sola prueba automatizada, en ninguno de los dos lenguajes. Ni **pytest**, ni **vitest**, ni **jest**. Cualquier cambio se validaba a ojo.

Lo que se hizo. Se añadió **frontend/scripts/prueba-agregaciones.mjs**, ejecutable con **npm run prueba:agregaciones**. Cubre exactamente la parte donde un error pasaría inadvertido: el filtrado y la agregación que ahora ocurren en el navegador. Son **41 comprobaciones** y tiene dos rasgos que la hacen valer:

1. **No usa datos inventados.** Corre contra los 3 958 eventos reales que sirve la API.
2. **Compara contra una fuente independiente.** Los valores esperados se calcularon aparte con pandas, no con el mismo código que se está probando. Comprobar que un código coincide consigo mismo no prueba nada.

Qué verifica, en grupos:

Grupo	Ejemplos de comprobación
Filtros simples	Veracruz → 392 eventos; septiembre → 684; Geológico → 302; Ciclones → 241
Filtrado cruzado	Omitir "tipos" devuelve las 3 958 filas; con Ciclones el ranking conserva los 9 tipos; sin omitir colapsaría a 1 barra
Integridad de agregados	La suma por estado cuadra con 668 035.68 MDP; los 12 meses suman 3 958; el histograma reparte los 3 958
Series temporales	La serie anual tiene 24 puntos y rellena con ceros los años sin eventos
Indicadores	3 958 eventos, 32 estados, 4 697 defunciones
Casos límite	Una combinación imposible (Yucatán + Sismos) da 0 eventos y no revienta ninguna función

Sale con código 1 si algo falla, así que sirve en integración continua.

Lo que sigue faltando, y hay que decirlo. Esta suite cubre el frontend. El **backend sigue sin pruebas**: no hay nada que verifique el contrato de **/predict**, la normalización de estados, la inversión **expm1**, ni que el orden de **FEATURES** siga coincidiendo entre **train_model_simple.py** y **main.py**. Tampoco hay una prueba que verifique el join de nombres entre **data.joblib** y **mexico.geojson** (ver L10).

Mejora propuesta. Una suite mínima de `pytest` sobre el backend:

```
# tests/test_contrato.py
def test_features_coinciden_entre_entrenamiento_y_api():
    """El fallo más caro del proyecto sería un desajuste silencioso de orden."""
    from app.main import FEATURES as api_features
    import json
    meta = json.load(open("app/artifacts/metadata.json"))
    assert api_features == meta["features"]

def test_prediccion_nunca_negativa():
    ...

def test_estado_desconocido_no_revienta():
    ...
```

L7. Sin versionado de modelos ni reentrenamiento programado

Evidencia. `train_model_simple.py` escribe siempre sobre los mismos seis nombres de archivo. `metadata.json` no contiene fecha de entrenamiento, ni identificador de versión, ni hash de los datos de origen. No hay tarea programada de reentrenamiento.

Consecuencia. Cuatro problemas prácticos:

- **No hay vuelta atrás.** Si un reentrenamiento produce un modelo peor, la versión anterior ya se sobrescribió.
- **No se sabe qué está en producción.** Dado un despliegue, no hay forma de determinar cuándo se entrenó ni con qué versión de `Base.xlsx`.
- **No se detecta la deriva.** Si el patrón de desastres cambia, nadie se entera.
- **Los datos envejecen.** La base termina en 2023. Los eventos de 2024, 2025 y 2026 no están, y el modelo servido no los conocerá nunca sin intervención manual.

Mejora propuesta, por etapas de costo creciente:

1. *Mínima:* añadir a `metadata.json` la marca de tiempo del entrenamiento, el hash SHA-256 de los dos Excel de origen y las versiones de `scikit-learn` y `numpy`. Son diez líneas de código y hacen auditable cualquier despliegue.
2. *Intermedia:* escribir los artefactos en `artifacts/<timestamp>/` y mantener `artifacts/latest` como enlace simbólico. La vuelta atrás pasa a ser cambiar el enlace.
3. *Completa:* MLflow o DVC para registro de experimentos, y una acción programada de GitHub que reentrene cuando cambien los archivos de origen, compare las métricas nuevas contra las vigentes y **rechace la promoción si el R^2 empeora** más allá de una tolerancia.

L8. Carga de artefactos en el import del módulo, sin manejo de fallos

Evidencia. El código real:

```
if MODEL_DANOS_PATH.exists() and PREP_PATH.exists():
    model_danos = joblib.load(MODEL_DANOS_PATH)
    preprocessor = joblib.load(PREP_PATH)
    ...
```

Sin `try/except`, sin `lifespan` de FastAPI, sin registro estructurado. La notificación de fallo es un `print()`:

```
print(f"[stats] {DATA_PATH.name} no encontrado. Ejecuta train_model_simple.py ...")
```

Consecuencia. Tres modos de fallo distintos, ninguno bien manejado:

- **Archivos ausentes:** las variables quedan en `None`, la API arranca aparentemente sana, `/health` devuelve **HTTP 200** con `"status": "ok"` aunque todos los indicadores estén en `false`, y `/predict` devuelve **HTTP 200** con una clave `"error"` en el cuerpo. Un monitor que vigile códigos de estado no detecta nada.
- **Archivos corruptos o de versión incompatible:** `joblib.load` lanza excepción **durante el import**, y el proceso muere antes de que Uvicorn levante. El servicio no arranca; en una plataforma con reinicio automático, entra en un ciclo de caídas.
- **Mensajes de error engañosos.** `_no_data()` devuelve literalmente *"Datos históricos no disponibles. Verifica la ruta del Excel."* — pero la API **no lee ningún Excel**. Ese mensaje es un residuo de la arquitectura anterior y mandaría a un operador a investigar donde no está el problema.

Mejora propuesta.

```

from contextlib import asynccontextmanager
from fastapi import HTTPException
import logging

log = logging.getLogger(__name__)
ESTADO = {"listo": False, "detalle": {}}

@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        cargar_artefactos()
        ESTADO["listo"] = True
    except Exception as exc:
        log.exception("No se pudieron cargar los artefactos")
        ESTADO["detalle"]["error"] = str(exc)
    yield

@app.get("/health")
def health():
    if not ESTADO["listo"]:
        raise HTTPException(status_code=503, detail=ESTADO["detalle"])
    ...

```

Tres cambios de fondo: usar `lifespan` para que un fallo no impida el arranque, devolver **503** cuando el servicio no puede cumplir su función, y sustituir `print` por `logging`. Y corregir el texto de `_no_data()`, que hoy es directamente incorrecto.

L9. Sin validación de dominio en las entradas

Evidencia verificada. El sistema acepta y responde a:

Entrada inválida	Respuesta
Mes: 99	389.49 MDP, 13 649 personas
Mes: -5 , Año: 1800	35.69 MDP, 445 personas
Tipo_de_fenómeno: "Tsunami"	1.71 MDP
Estado: "Narnia"	126.37 MDP

Consecuencia. El sistema responde con total aplomo a preguntas sin sentido. Es especialmente insidioso con estados y fenómenos: gracias a `handle_unknown="ignore"`, un error de escritura como `"Guerero"` no produce error, produce una predicción **basada en un vector con el bloque de estado en ceros** — es decir, una predicción degradada que el usuario no tiene forma de distinguir de una válida.

Mejora propuesta. Pydantic v2 ofrece exactamente lo que hace falta:

```

from typing import Literal
from pydantic import BaseModel, Field

ESTADOS = Literal["Aguascalientes", "Baja California", ..., "Zacatecas"]
TIPOS = Literal["Actividad Volcánica", "Sismos", ..., "Calor Extremo"]

class PredictRequest(BaseModel):
    Año: int = Field(ge=2000, le=2030)
    Mes: int = Field(ge=1, le=12)
    Clasificación_del_fenómeno: Literal["Geológico", "Hidrometeorológico"]
    Tipo_de_fenómeno: TIPOS
    Estado: ESTADOS

```

Con esto, una entrada inválida produce **HTTP 422** con un mensaje que dice exactamente qué campo está mal — el comportamiento correcto. Idealmente las listas se derivarían de `preprocessor.named_transformers_["cat"].categories_` para que no puedan divergir del modelo. Convendría además advertir explícitamente cuando `Año > 2023`, ya que la predicción se congela en el último año entrenado.

L10. Riesgo latente de desalineación de nombres de estado

Evidencia. El join está **completo hoy** (verificado: 32 = 32, cero faltantes en ambas direcciones). Pero descansa en la coincidencia exacta de cadenas entre cuatro artefactos mantenidos por separado:

Artefacto	Diccionario	Ubicación
<code>Base.xlsx</code>	—	raíz del repositorio
<code>Poblacion_01.xlsx</code>	<code>CENSUS_TO_BASE</code>	<code>train_model_simple.py</code>
API	<code>STATE_ALIASES</code>	<code>main.py</code> (duplicado de <code>train_model_simple.py</code>)
<code>mexico.geojson</code>	<code>GEO_NAME_MAP</code>	<code>DashboardHistorico.jsx</code>

Consecuencia. Tres diccionarios de traducción distintos, en tres archivos, en dos lenguajes de programación. `STATE_ALIASES` está **literalmente duplicado** entre el entrenamiento y la API — dos definiciones que deben mantenerse iguales sin nada que lo garantice. Si el GeoJSON se reemplazara por uno con nombres distintos, el mapa se pintaría **completamente gris** sin lanzar un solo error, y solo una inspección visual lo detectaría.

Mejora propuesta.

1. Extraer la normalización a un módulo compartido `backend/app/estados.py` con una única definición de alias, importado tanto por el entrenamiento como por la API.
2. Publicar un endpoint `GET /stats/estados` que devuelva la lista canónica, y que el frontend construya su selector desde ahí en lugar de tener 32 `<option>` escritos a mano.

3. Extender `verifica-geojson.mjs` para que además del área de los anillos compruebe que **todo nombre del GeoJSON, tras aplicar `GEO_NAME_MAP`, existe en la respuesta de `/stats/por-estado`**, y salga con código 1 si no.

L11. Sesgo de retransformación: subestimación sistemática

Evidencia medida. Sobre las 732 filas de prueba:

Magnitud	Valor
Suma real del daño	148 322.2 MDP
Suma de las predicciones	13 131.7 MDP
Cobertura	8.9 %

Y en el modelo de población, predicciones como 161 personas para un ciclón en Guerrero, cuando la mediana histórica del objetivo es 85 y el percentil 90 es 10 045.

Consecuencia. Las predicciones son sistemáticamente bajas y **no pueden sumarse**. Para el modelo de población el efecto es visualmente evidente: la interfaz muestra cifras de dos y tres dígitos que a cualquier usuario le parecerán —con razón— demasiado pequeñas para un desastre.

Mejora propuesta. Dos caminos, ambos viables:

1. **Factor de suavizado de Duan.** Corrección clásica y barata: se calcula sobre el conjunto de entrenamiento el promedio de `exp(residuo)` en escala log, y se multiplica la predicción retransformada por ese factor. Corrige el sesgo del agregado sin cambiar la arquitectura.
2. **Modelo en dos etapas (*hurdle model*).** Más apropiado para la estructura real de los datos: un clasificador que estime `P(daño > 0)` —recuérdese que el 26.9 % de los daños son exactamente cero— y un regresor entrenado solo sobre los positivos. La predicción final es el producto. Modela explícitamente la masa en cero, que hoy el regresor único tiene que absorber distorsionando todo lo demás.

Sea cual sea la elección, **la interfaz debe advertir que la estimación es conservadora**, cosa que hoy no hace.

L12. Datos simulados en el dashboard — *resuelta*

Situación original. Seis constantes `MOCK_*` con cifras inventadas, usadas como valor inicial de `useState`. Si la API no respondía, el dashboard mostraba 1 842 eventos y 524 300.5 MDP —números falsos— junto a un mensaje de error.

Por qué importaba. Era el riesgo de credibilidad más serio de todo el frontend. En una demostración en vivo con la API caída, el panel se veía completo y funcional con datos que no existen, y las cifras falsas eran plausibles: del orden de magnitud correcto.

Indicador	Valor simulado que se mostraba	Valor real
Total de eventos	1 842	3 958
Daño total	524 300.5 MDP	668 035.68 MDP
Población afectada	12 450 000	58 897 609

Lo que se hizo. Se eliminaron las seis constantes. El componente renderiza ahora tres estados mutuamente excluyentes —cargando, error, con datos— y en ninguno de ellos puede aparecer una cifra que no venga de la API. El estado de error nombra explícitamente el valor de `VITE_API_URL`, que es el dato que hace falta para diagnosticar el fallo.

L13. Sin optimización de hiperparámetros

Evidencia. `HistGradientBoostingRegressor(random_state=42)`, todo lo demás por omisión. No hay `GridSearchCV`, `RandomizedSearchCV` ni `Optuna`.

Consecuencia. Es probable que exista margen sin explotar. Con solo 3 958 filas y 47 columnas, `max_iter=100` y `max_leaf_nodes=31` podrían no ser óptimos, y `l2_regularization=0.0` deja la regularización enteramente en manos de `min_samples_leaf`.

Mejora propuesta. Una búsqueda aleatoria con **validación cruzada respetando el orden temporal** (`TimeSeriesSplit`, nunca `KFold` mezclado, por lo explicado en el capítulo 6) sobre `learning_rate`, `max_iter`, `max_leaf_nodes`, `min_samples_leaf` y `l2_regularization`:

```
from sklearn.model_selection import RandomizedSearchCV, TimeSeriesSplit
busqueda = RandomizedSearchCV(
    HistGradientBoostingRegressor(random_state=42),
    {"learning_rate": [0.01, 0.05, 0.1], "max_iter": [100, 300, 600],
     "max_leaf_nodes": [15, 31, 63], "min_samples_leaf": [10, 20, 40],
     "l2_regularization": [0.0, 0.1, 1.0]},
    n_iter=40, cv=TimeSeriesSplit(n_splits=4), random_state=42,
)
```

Expectativa honesta: una ganancia probable pero modesta, quizá unas décimas de R^2 . El techo real lo pone la ausencia de variables de intensidad (L4), no la configuración del algoritmo. Conviene decirlo así en la defensa: ajustar hiperparámetros es lo que se hace cuando ya no hay mejores datos que conseguir, y aquí sí los hay.

L14. Simplificación de eventos multiestado

Evidencia. `df["Estado"].str.split(r",| y ").str[0]` conserva solo el primer estado. En la base hay 5 valores multiestado distintos, incluidos `"CDMX, Estado de Mexico, Morelos, Puebla, Tlaxcala"` (cinco entidades) y `"Chihuahua, Durango, Estado de Mexico, Puebla, Sonora, Tlaxcala"` (seis). Además se eliminan 2 filas con la etiqueta `"Varios Estados"`.

Consecuencia. El daño de los estados secundarios desaparece de las estadísticas y del mapa. El primer estado listado —que a menudo lo es por orden alfabético, no por severidad— recibe el 100 % del daño atribuido.

Mejora propuesta. Expandir cada fila multiestado en varias, una por entidad, y repartir el daño. El reparto puede ser proporcional a la población estatal —una aproximación razonable a la exposición— añadiendo una columna `peso_reparto` para que las agregaciones no dupliquen. Afecta a pocas filas, pero elimina un sesgo sistemático y es de implementación sencilla.

L15. Ceros ambiguos en `Población afectada`

Evidencia. 466 filas (11.9 %) con valor exactamente 0 y 44 filas con el literal `"SD"`.

Consecuencia. Un cero puede significar dos cosas incompatibles: "el evento no afectó a nadie" o "nadie contabilizó a los afectados". El modelo trata ambos como la primera. Dado que la mediana del objetivo es 85 personas, esos 466 ceros arrastran las predicciones hacia abajo y explican en parte el R^2 log de 0.1033, el más bajo de los dos modelos.

Mejora propuesta. Auditar los ceros cruzándolos con el resto de columnas de consecuencias: si una fila tiene `Población afectada = 0` pero `Viviendas dañadas > 0` o `Defunciones > 0`, el cero es casi con seguridad un dato faltante mal codificado y debería convertirse en `NaN`. Es una regla de consistencia interna que no requiere ninguna fuente externa. La columna `Descripcion general de los daños` permitiría además una verificación cualitativa por muestreo.

L16. Escala lineal de color en el mapa — *resuelta*

Situación original. `scaleLinear().domain([0, maxVal])` con `maxVal` = 118 142 MDP (Guerrero), sobre una rampa de rojo.

Por qué importaba. Con el 66.7 % del daño concentrado en el 1 % de eventos, la distribución del daño acumulado por estado también es muy sesgada. Sinaloa, décimo del ranking con 14 458 MDP, se pintaba al 12 % de la escala. La mitad inferior de los estados era visualmente indistinguible.

Lo que se hizo. Escala en **raíz cuadrada** sobre una rampa discreta de ocho pasos de un solo tono, con leyenda escalonada que hace explícita la correspondencia entre color y valor. La raíz cuadrada comprime la cola sin aplastar el extremo bajo como haría un logaritmo puro, y admite el cero sin caso especial.

Lo que queda abierto. `scaleQuantile` repartiría los estados en grupos de igual tamaño y garantizaría contraste máximo entre clases adyacentes. Es defendible como alternativa; la raíz cuadrada se prefirió porque conserva la noción de magnitud absoluta (un estado con el doble de daño se ve más oscuro), mientras que la escala por cuantiles solo conserva el orden.

L17. Deuda técnica menor pero real

Cuatro puntos que conviene tener identificados:

Punto	Evidencia	Corrección
<code>req.dict()</code> está obsoleto	Pydantic 2.9.2 emite <code>PydanticDeprecatedSince20</code> ; se eliminará en Pydantic v3	Cambiar a <code>req.model_dump()</code>
<code>robocopy</code> solo existe en Windows	<code>_read_excel_safe</code> invoca un ejecutable ausente en Linux; el respaldo lanzaría <code>FileNotFoundError</code> en lugar del <code>PermissionError</code> descriptivo	Envolver en <code>try/except</code> <code>FileNotFoundError</code> o condicionar a <code>sys.platform == "win32"</code>
<code>legacy-peer-deps=true</code>	<code>react-simple-maps@3</code> declara React 16-18; el proyecto usa React 19	Migrar a una biblioteca de mapas compatible con React 19, o fijar la versión y documentar la excepción
Sin enrutador en el frontend	Las dos pestañas son un condicional sobre <code>useState</code> ; no hay URL propia ni historial de navegación	Añadir <code>react-router</code> si se necesitan enlaces directos; hoy es una omisión defendible

Qué haría falta para llevarlo a producción real

Recogiendo lo anterior en un orden de prioridad operativa:

Bloqueantes (sin esto no debería exponerse a usuarios reales):

- Validación de dominio con `Literal` y `Field` — **L9**.
- Manejo de fallos en la carga de artefactos y `/health` que devuelva 503 — **L8**.
- Eliminar los datos simulados del dashboard — **L12**.
- Intervalos de predicción, o como mínimo una advertencia visible de incertidumbre — **L3**.
- Autenticación y limitación de tasa en la API.

Importantes (determinan si el sistema es útil o solo funcional):

- Variables de intensidad del evento, empezando por ciclones — **L4**.
- Granularidad municipal — **L2**.
- Corrección del sesgo de retransformación — **L11**.
- Conjunto de pruebas de contrato — **L6**.
- Versionado de artefactos con marca de tiempo y hash de origen — **L7**.

Deseables (mejoran la calidad y el mantenimiento):

- Deflactar por INPC — **L5**.
- Reentrenamiento programado con promoción condicionada a métricas — **L7**.
- Búsqueda de hiperparámetros con `TimeSeriesSplit` — **L13**.
- Escala de color no lineal en el mapa — **L16**.
- Módulo compartido de normalización de estados — **L10**.

Infraestructura, transversal a todo lo anterior: registro estructurado, monitoreo de la latencia y la tasa de error, alertas sobre `/health`, y un registro de las predicciones servidas para poder evaluar el desempeño real frente a los eventos que efectivamente ocurran.

CAPÍTULO 11

Glosario

Términos ordenados para consulta rápida, redactados para poder explicarlos a alguien sin formación técnica.

Aprendizaje supervisado. Familia de métodos en los que el modelo aprende a partir de ejemplos que ya traen la respuesta. Aquí, cada fila del Excel es un ejemplo: las características del evento y el daño que efectivamente causó.

Artefacto (*artifact*). Archivo producido por el entrenamiento que el servicio carga después. En este proyecto son los seis archivos de `backend/app/artifacts/`.

Asimetría (*skewness*). Medida de cuán inclinada está una distribución. Cero indica simetría. El daño económico tiene asimetría 29.61: casi todos los valores están pegados al cero y unos pocos se disparan.

Cola pesada (*heavy tail*). Distribución en la que los valores extremos son mucho más frecuentes de lo que una campana de Gauss predeciría. Es la forma característica de las pérdidas por desastre.

CORS (*Cross-Origin Resource Sharing*). Mecanismo por el que un navegador decide si una página de un sitio puede pedir datos a otro sitio. Sin una autorización explícita del servidor, el navegador bloquea la petición.

Curtosis. Medida de cuánto peso tiene la distribución en sus extremos. Una normal tiene 0 (en la definición de exceso); el daño económico tiene 1 131.5.

Ex-ante / ex-post. *Ex-ante*: antes del hecho. *Ex-post*: después. Este sistema es ex-ante: estima antes de que el daño se conozca.

Feature (variable de entrada, característica). Cada dato que el modelo recibe para predecir. Aquí son siete tras el preprocesamiento: tres categóricas y cuatro numéricas.

Fuga de datos (*data leakage*). Error metodológico en el que el modelo recibe información que en el momento real de uso no estaría disponible —a menudo porque se deriva de la propia respuesta—. Produce métricas excelentes y un sistema inservible. Evitarla es el punto fuerte de este proyecto.

GeoJSON. Formato de archivo, basado en JSON, para describir geometrías geográficas. `mexico.geojson` contiene los polígonos de los 32 estados.

Gradient boosting (potenciación del gradiente). Técnica que construye una secuencia de modelos simples —árboles de decisión pequeños— donde cada uno se entrena para corregir el error que dejaron los anteriores. La suma de todos es el modelo final. Analogía: en lugar de un experto que decide solo, un comité donde cada miembro se especializa en los casos que el comité aún falla.

HistGradientBoostingRegressor. Implementación de gradient boosting en scikit-learn que agrupa los valores numéricos en 256 cubetas (histogramas) antes de buscar los puntos de corte, lo que la hace mucho más rápida.

Hiperparámetro. Ajuste que se fija *antes* de entrenar (número de árboles, tasa de aprendizaje), a diferencia de los parámetros que el modelo aprende de los datos.

Interpolación. Estimar un valor intermedio entre dos conocidos. Aquí, la población de 2013 se calcula como un punto sobre la recta entre los censos de 2010 y 2020.

Joblib. Biblioteca de Python para guardar y recuperar objetos —incluidos modelos de scikit-learn y `DataFrames`— en archivos. Es la que produce los `.joblib`.

Línea base (*baseline*). Modelo trivial contra el que se compara para saber si el modelo real aporta algo. Aquí: "predecir siempre la mediana".

log1p / expm1. `log1p(y) = ln(1+y)` comprime una escala muy amplia; `expm1(x) = ex-1` la descomprime. Se usan en pareja: la primera antes de entrenar, la segunda al predecir. La variante "1p" permite manejar el valor cero, que el logaritmo ordinario no admite.

MAE (*Mean Absolute Error, error absoluto medio*). Promedio de la distancia entre lo predicho y lo real, sin signo. Está en las unidades del objetivo: un MAE de 209.09 significa que el modelo se equivoca, en promedio, por 209.09 millones de pesos.

MedAE (*Median Absolute Error, error absoluto mediano*). Lo mismo pero usando la mediana en lugar del promedio. Es robusto a los valores extremos: por eso aquí vale 1.69 mientras el MAE vale 209.09.

One-hot encoding (codificación de una entre N). Convertir una variable categórica en varias columnas binarias, una por categoría, con un 1 en la que corresponde. Evita sugerirle al modelo un orden entre categorías que no lo tiene.

Overfitting (sobreajuste). Cuando el modelo memoriza el conjunto de entrenamiento —incluido su ruido— en vez de aprender el patrón general. Se detecta porque acierta mucho en entrenamiento y poco en datos nuevos.

Preprocesador. Objeto que transforma los datos crudos al formato numérico que el modelo necesita. Aquí, un `ColumnTransformer` que aplica one-hot a tres columnas y deja pasar cuatro sin tocar.

R² (coeficiente de determinación). Proporción de la variabilidad del objetivo que el modelo explica. 1 es perfecto; 0 equivale a predecir siempre el promedio; **negativo significa peor que predecir el promedio**. En este proyecto se calcula sobre el logaritmo del objetivo, por eso se anota `R2log`.

Split temporal. Repartir datos en entrenamiento y prueba **por fecha** —pasado para entrenar, futuro para evaluar— en lugar de al azar. Es obligatorio cuando el modelo se usará para predecir hacia adelante.

Validación cruzada. Repetir el entrenamiento y la evaluación con distintas particiones de los datos para obtener una métrica más estable. En datos temporales debe respetar el orden (`TimeSeriesSplit`).

Variables cíclicas. Codificar una magnitud circular —el mes, la hora, el ángulo— como el seno y el coseno de su posición en la circunferencia, para que el primer y el último valor queden vecinos.

CAPÍTULO 12

Referencias

Fuentes de datos

- **CENAPRED — Centro Nacional de Prevención de Desastres.** Serie *Impacto socioeconómico de los desastres en México*. Origen de `Base.xlsx`, junto con registros de FONDEN, CENACOM, SAGARPA, PACC, FAPRACC y CEPAL identificados en su columna `Fuente`. <https://www.gob.mx/cenapred>
- **INEGI — Instituto Nacional de Estadística y Geografía.** Tabulado *Población total por entidad federativa y grupo quinquenal de edad según sexo, serie de años censales de 1990 a 2020*. Origen de `Poblacion_01.xlsx`; fecha de consulta registrada en el archivo: 13/06/2026. <https://www.inegi.org.mx>
- **INEGI.** Marco Geoestadístico Nacional, del que derivan las geometrías estatales de `mexico.geojson`.

Bibliotecas y herramientas

- **scikit-learn 1.5.1.** Pedregosa, F. *et al.* (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. Documentación de `HistGradientBoostingRegressor`, `ColumnTransformer` y `OneHotEncoder`: <https://scikit-learn.org/stable/>
- **FastAPI 0.115.0.** Ramírez, S. Framework web asíncrono con validación Pydantic y generación automática de OpenAPI. <https://fastapi.tiangolo.com>
- **Pydantic 2.9.2.** Validación de datos por anotaciones de tipo. <https://docs.pydantic.dev>
- **pandas 2.2.2.** McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. <https://pandas.pydata.org>
- **NumPy 2.0.1.** Harris, C. R. *et al.* (2020). *Array programming with NumPy*. Nature 585, 357–362.
- **joblib 1.4.2.** Serialización eficiente de objetos de Python.
- **Uvicorn 0.30.6.** Servidor ASGI.
- **React 19.** <https://react.dev>
- **Vite 7.** <https://vite.dev>
- **react-simple-maps 3.** Componentes de mapa para React sobre d3-geo. <https://www.react-simple-maps.io>
- **D3 (d3-scale, d3-geo).** Bostock, M. *et al.* (2011). *D3: Data-Driven Documents*. IEEE Transactions on Visualization and Computer Graphics.
- **RFC 7946.** *The GeoJSON Format*. IETF, 2016. Define la convención de orientación de anillos que verifica `verifica-geojson.mjs`.

Fundamentos metodológicos

- **Gradient boosting.** Friedman, J. H. (2001). *Greedy Function Approximation: A Gradient Boosting Machine*. Annals of Statistics, 29(5), 1189–1232.

- **Boosting por histogramas.** Ke, G. *et al.* (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS 30. Base algorítmica de `HistGradientBoostingRegressor`.
- **Sesgo de retransformación.** Duan, N. (1983). *Smearing Estimate: A Nonparametric Retransformation Method*. Journal of the American Statistical Association, 78(383), 605–610. Fundamento de la corrección propuesta en L11.
- **Validación en series temporales.** Bergmeir, C. y Benítez, J. M. (2012). *On the use of cross-validation for time series predictor evaluation*. Information Sciences, 191, 192–213. Sustento del split temporal del capítulo 6.
- **Árboles frente a redes en datos tabulares.** Grinsztajn, L., Oyallon, E. y Varoquaux, G. (2022). *Why do tree-based models still outperform deep learning on typical tabular data?* NeurIPS Datasets and Benchmarks.

Documentación interna del proyecto

- `CLAUDE.md` — arquitectura y convenciones del repositorio.
- `docs/superpowers/specs/2026-06-13-modelos-ex-ante-design.md` — especificación de diseño de los modelos ex-ante, con el diagnóstico de la fuga de datos y los criterios de aceptación.
- `backend/app/artifacts/metadata.json` — métricas del modelo entregado.
- Historial de Git del repositorio `sacalvario/Desastres-naturales`, rama `main`.

Nota sobre la verificación de las cifras

Todas las cifras de este manual —conteos de filas, distribuciones, percentiles, métricas y resultados de los experimentos comparativos— se obtuvieron ejecutando código sobre los archivos reales del repositorio en la fecha de elaboración: `Base.xlsx`, `Poblacion_01.xlsx`, los seis artefactos de `backend/app/artifacts/` y `frontend/public/mexico.geojson`.

Los experimentos que no forman parte del código del proyecto —la comparación con Ridge, con el objetivo sin transformar, con el split aleatorio y con las columnas de fuga— se realizaron específicamente para este documento, reutilizando las funciones de `train_model_simple.py` sin modificarlas, con `random_state=42` para que sean reproducibles.

Ninguna cifra de este manual es estimada, redondeada de memoria o inventada.